UPLA

Construcción de software

Semana 03
(Laboratorio)

Analizar y aplicar estándares de construcción de software.

ASIGNATURA
Construcción de software

ACTIVIDAD 01

“La simplicidad es la máxima sofisticación.”

— Leonardo da Vinci

1. **Objetivo**

   Analizar y aplicar estándares de construcción de software mediante la definición e implementación inicial de la arquitectura del sistema ConstruSoft Ticket, utilizando el stack tecnológico definido.

2. **Contexto**

   El equipo de desarrollo ha recibido la responsabilidad de iniciar la construcción del sistema ConstruSoft Ticket, orientado a la gestión de incidencias técnicas en el laboratorio de cómputo.

   Sin embargo, existe un problema recurrente en proyectos de software: muchos sistemas comienzan sin una estructura clara, lo que genera desorden, dificultad de mantenimiento y baja calidad del producto final.

   Para evitar estos problemas, el equipo ha decidido aplicar desde el inicio principios de arquitectura y estándares de construcción, considerando enfoques como:

   - Separación de responsabilidades.
   - Organización por capas.
   - Principios SOLID.
   - Código limpio.
   - Integración de frontend, backend y base de datos.

Tal como plantean Roger S. Pressman y Ian Sommerville, la correcta aplicación de estándares en las etapas iniciales del desarrollo permite mejorar la calidad, escalabilidad y mantenibilidad del software.

3. Problema de la semana

El equipo debe iniciar el desarrollo del sistema ConstruSoft Ticket, pero enfrenta la siguiente situación:

¿Cómo estructurar e implementar la base del sistema asegurando que sea escalable, mantenible y alineado a buenas prácticas de construcción de software?

4. Condiciones y restricciones

Para resolver el problema, se deben considerar las siguientes condiciones:

a) El sistema debe desarrollarse utilizando:
   - C# (backend)
   - React con Vite (frontend)
   - PostgreSQL (base de datos)
   - Visual Studio Code (entorno de desarrollo)

b) Se debe aplicar un enfoque basado en:
   - Arquitectura por capas (referencia a Clean Architecture)
   - Principios SOLID
   - Organización modular del código

c) El sistema debe permitir, como mínimo:
   - Definir una entidad principal del sistema
   - Proponer un primer caso de uso
   - Iniciar una interfaz básica de interacción

5. Actividades / Preguntas orientadoras

El estudiante deberá analizar y proponer una solución respondiendo:

1. ¿Cómo estructuraría el backend del sistema para asegurar separación de responsabilidades?
2. ¿Qué capas considera necesarias y cuál sería la función de cada una?
3. ¿Cómo organizaría el frontend para que sea escalable?
4. ¿Qué entidad principal tendría el sistema y qué atributos incluiría?
5. ¿Qué primer caso de uso implementaría y por qué?
6. ¿Cómo se conectaría el sistema con la base de datos?
7. ¿Qué principios SOLID podría aplicar en esta etapa inicial?
8. ¿Qué estándares de codificación considera importantes desde el inicio?

6. Entregable

El estudiante deberá presentar:

1. Propuesta de arquitectura del sistema (puede ser diagrama o explicación estructurada)
2. Estructura de carpetas del proyecto (backend y frontend)
3. Definición de la entidad principal del sistema
4. Descripción del primer caso de uso
5. Avance inicial del sistema (código base)
6. Documento breve (1–2 páginas) justificando:
    - Decisiones de arquitectura
    - Estándares aplicados
    - Principios utilizados

7. Reflexión final

- ¿Qué diferencia existe entre iniciar un sistema sin arquitectura y con una arquitectura definida?
- ¿Cómo influyen los estándares en la calidad del software?
- ¿Considera que su propuesta facilitará el desarrollo en las siguientes semanas? ¿Por qué?

Solución (modelo)

Solución modelo: Inicio del desarrollo del sistema ConstruSoft Ticket

1. **Propósito de la actividad**

   Orientar al estudiante en la implementación inicial del Sistema ConstruSoft Ticket, aplicando una estructura basada en Clean Architecture con el stack definido para la asignatura:

   - Backend: C# con ASP.NET Core
   - Frontend: React con Vite
   - Base de datos: PostgreSQL
   - Entorno de desarrollo: Visual Studio Code

   La actividad busca que el estudiante construya una base técnica ordenada, mantenible y escalable, alineada al desarrollo progresivo del curso y al sílabo 2026-I.

2. **Análisis del problema**

   El problema de la semana consiste en evitar que el sistema se inicie de manera improvisada. La solución técnica recomendada es construir desde el inicio una base organizada, con separación de responsabilidades y criterios de calidad que faciliten el crecimiento del sistema en las semanas siguientes.

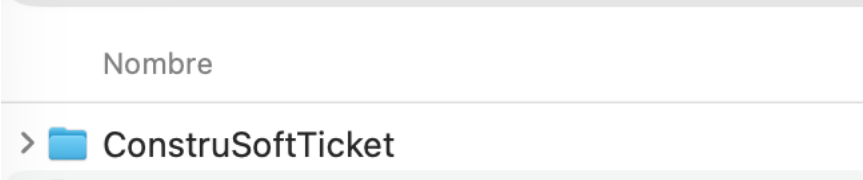
   Por ello, esta guía propone:

   - Una estructura backend basada en capas,
   - Una organización inicial del frontend,
   - Una primera entidad del dominio,
   - Un primer caso de uso,
   - Y una primera interfaz básica.

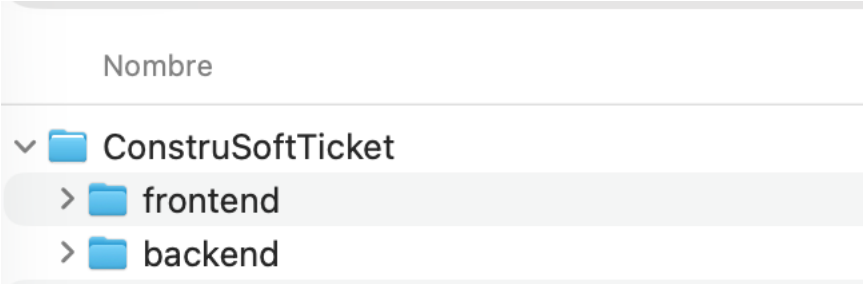
3. **Estructura general esperada**

   3.1. Carpeta principal del proyecto

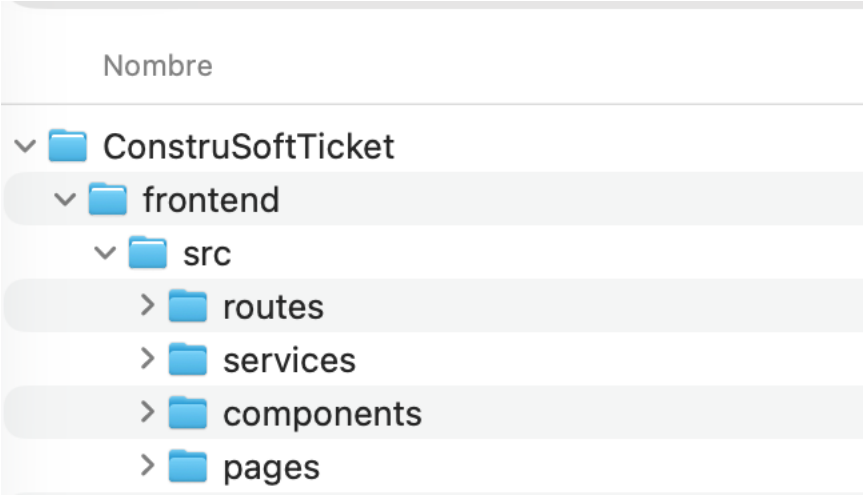
   La solución debe organizarse en una carpeta principal llamada:



Dentro de ella se recomienda trabajar con esta estructura inicial:



3.2. Estructura esperada del backend



4. Desarrollo paso a paso reproducible

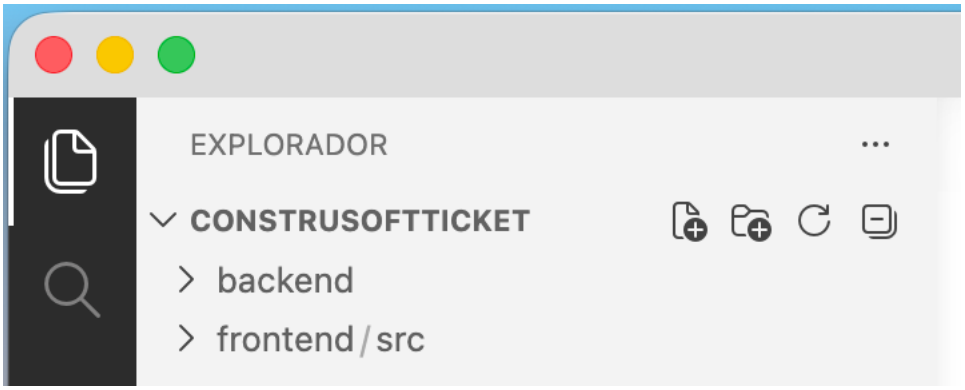
Paso 0. Crear la carpeta raíz del proyecto.

Antes de ejecutar comandos de .NET o React, el docente debe indicar a los estudiantes que creen la carpeta principal del proyecto y la abran en Visual Studio Code.

Elemento	macOS (MacBook Pro M2)	Windows 11
Acción	Crear la carpeta ConstruSoftTicket en una ubicación de trabajo, por ejemplo en Documentos o Escritorio.	Crear la carpeta ConstruSoftTicket en una ubicación de trabajo, por ejemplo en Documentos o Escritorio.
Recomendación	Evitar rutas con espacios o caracteres especiales.	Evitar rutas con espacios o caracteres especiales.

Apertura en VS Code	Abrir VS Code y seleccionar File > Open Folder.	Abrir VS Code y seleccionar File > Open Folder.
Resultado esperado	VS Code debe mostrar la carpeta ConstruSoftTicket como proyecto abierto.	VS Code debe mostrar la carpeta ConstruSoftTicket como proyecto abierto.

Estructura a crear manualmente



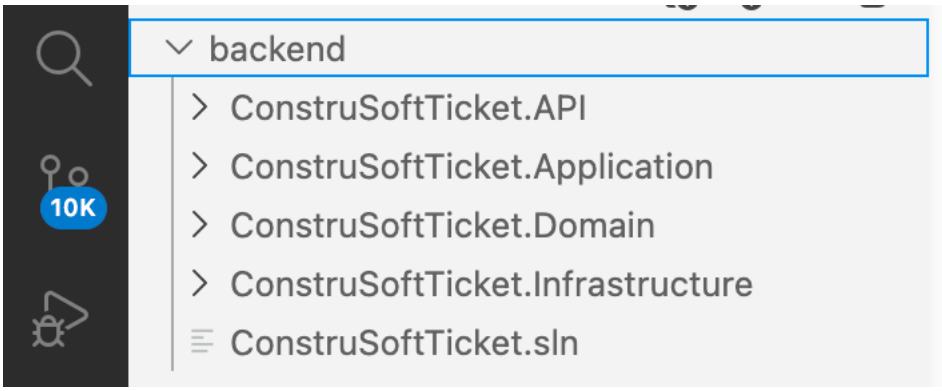
Paso 1. Crear la solución .NET.

En este paso ya no basta con dar el comando; hay que indicar dónde ejecutarlo.

Aspecto	macOS	Windows 11
Herramienta a usar	Terminal integrada de VS Code	Terminal integrada de VS Code
Carpeta desde donde se ejecuta	Ingresar a la carpeta backend.	Ingresar a la carpeta backend.
Comando para ubicarse	cd backend	cd backend
Comando para crear la solución	dotnet new sln -n ConstruSoftTicket	dotnet new sln -n ConstruSoftTicket
Resultado esperado	Se crea el archivo ConstruSoftTicket.sln dentro de backend.	Se crea el archivo ConstruSoftTicket.sln dentro de backend.

Verificación esperada

Al terminar este paso, la carpeta backend debe contener al menos:

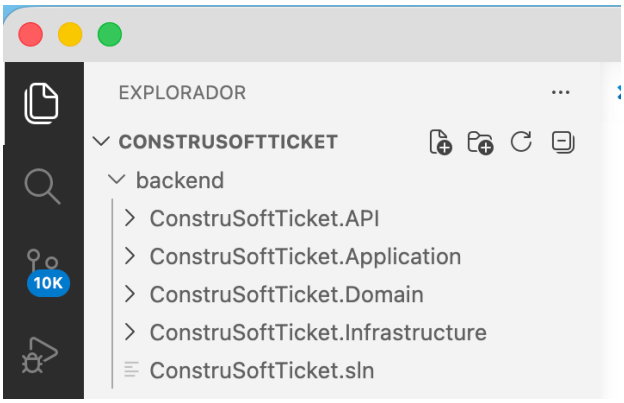


Paso 2. Crear los proyectos del backend.

Todos los comandos de este paso también deben ejecutarse dentro de la carpeta backend.

Acción	macOS	Windows 11	Resultado esperado
Verificar ubicación.	pwd	cd	Debe apuntar a la carpeta backend.
Crear Domain	dotnet new classlib -n ConstruSoftTicket.Domain	dotnet new classlib -n ConstruSoftTicket.Domain	Se crea la carpeta ConstruSoftTicket.Domain
Crear Application	dotnet new classlib -n ConstruSoftTicket.Application	dotnet new classlib -n ConstruSoftTicket.Application	Se crea la carpeta ConstruSoftTicket.Application
Crear Infrastructure	dotnet new classlib -n ConstruSoftTicket.Infrastructure	dotnet new classlib -n ConstruSoftTicket.Infrastructure	Se crea la carpeta ConstruSoftTicket.Infrastructure
Crear API	dotnet new webapi -n ConstruSoftTicket.API	dotnet new webapi -n ConstruSoftTicket.API	Se crea la carpeta ConstruSoftTicket.API

Estructura esperada tras este paso



Paso 3. Agregar los proyectos a la solución.

Este paso también se realiza desde la carpeta backend.

Opción recomendada para evitar diferencias entre sistemas

En lugar de usar comodines que pueden comportarse distinto según el shell, conviene usar comandos explícitos.

Acción	macOS	Windows 11	Resultado esperado
Ubicarse en backend	cd backend	cd backend	Terminal en la carpeta correcta.
Agregar Domain	dotnet sln ConstruSoftTicket.sln add ConstruSoftTicket.Domain/ConstruSoftTicket.Domain.csproj	dotnet sln ConstruSoftTicket.sln add ConstruSoftTicket.Domain\ConstruSoftTicket.Domain.csproj	Proyecto agregado
Agregar Application	dotnet sln ConstruSoftTicket.sln add ConstruSoftTicket.Application/ConstruSoftTicket.Application.csproj	dotnet sln ConstruSoftTicket.sln add ConstruSoftTicket.Application\ConstruSoftTicket.Application.csproj	Proyecto agregado
Agregar Infrastructure	dotnet sln ConstruSoftTicket.sln add ConstruSoftTicket.Infrastructure/ConstruSoftTicket.Infrastructure.csproj	dotnet sln ConstruSoftTicket.sln add ConstruSoftTicket.Infrastructure\ConstruSoftTicket.Infrastructure.csproj	Proyecto agregado
Agregar API	dotnet sln ConstruSoftTicket.sln add ConstruSoftTicket.API/ConstruSoftTicket.API.csproj	dotnet sln ConstruSoftTicket.sln add ConstruSoftTicket.API\ConstruSoftTicket.API.csproj	Proyecto agregado

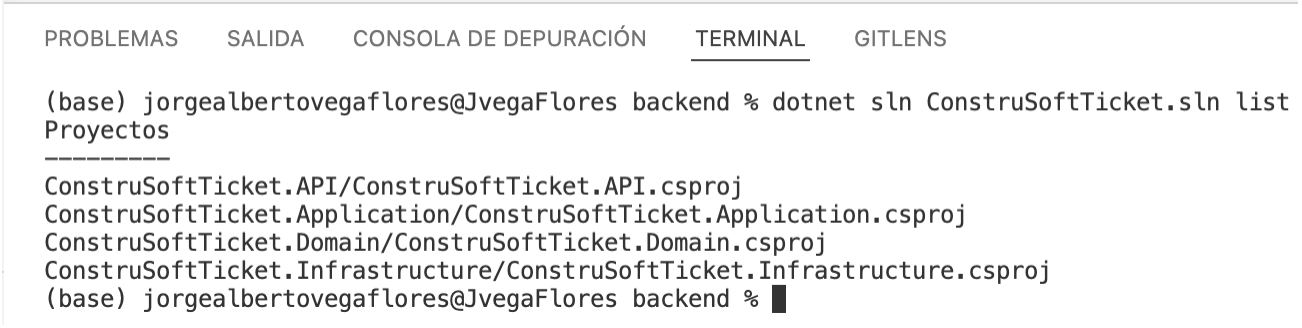
Verificación

Ejecutar:

macOS	Windows 11
dotnet sln ConstruSoftTicket.sln list	dotnet sln ConstruSoftTicket.sln list

Resultado esperado

La solución debe listar los cuatro proyectos.



Paso 4. Configurar las referencias entre proyectos

Este paso también debe hacerse desde la carpeta backend.

Relación esperada

- ConstruSoftTicket.Application debe referenciar a ConstruSoftTicket.Domain
- ConstruSoftTicket.Infrastructure debe referenciar a ConstruSoftTicket.Application y ConstruSoftTicket.Domain
- ConstruSoftTicket.API debe referenciar a ConstruSoftTicket.Application e ConstruSoftTicket.Infrastructure

Acción	macOS	Windows 11	Resultado esperado
Application referencia a Domain.	dotnet add ConstruSoftTicket.Application/ConstruSoftTicket.Application.csproj reference ConstruSoftTicket.Domain/ConstruSoftTicket.Domain.csproj	dotnet add ConstruSoftTicket.Application\ConstruSoftTicket.Application.csproj reference ConstruSoftTicket.Domain\ConstruSoftTicket.Domain.csproj	Referencia agregada
Infrastructure referencia a Application.	dotnet add ConstruSoftTicket.Infrastructure/ConstruSoftTicket.Infrastructure.csproj reference ConstruSoftTicket.Application/ConstruSoftTicket.Application.csproj	dotnet add ConstruSoftTicket.Infrastructure\ConstruSoftTicket.Infrastructure.csproj reference ConstruSoftTicket.Application\ConstruSoftTicket.Application.csproj	Referencia agregada
Infrastructure referencia a Domain	dotnet add ConstruSoftTicket.Infrastructure/ConstruSoftTicket.Infrastructure.csproj reference ConstruSoftTicket.Domain/ConstruSoftTicket.Domain.csproj	dotnet add ConstruSoftTicket.Infrastructure\ConstruSoftTicket.Infrastructure.csproj reference ConstruSoftTicket.Domain\ConstruSoftTicket.Domain.csproj	Referencia agregada
API referencia a Application	dotnet add ConstruSoftTicket.API/ConstruSoftTicket.API.csproj reference ConstruSoftTicket.Application/ConstruSoftTicket.Application.csproj	dotnet add ConstruSoftTicket.API\ConstruSoftTicket.API.csproj reference ConstruSoftTicket.Application\ConstruSoftTicket.Application.csproj	Referencia agregada
API referencia a Infrastructure	dotnet add ConstruSoftTicket.API/ConstruSoftTicket.API.csproj reference ConstruSoftTicket.Infrastructure/ConstruSoftTicket.Infrastructure.csproj	dotnet add ConstruSoftTicket.API\ConstruSoftTicket.API.csproj reference ConstruSoftTicket.Infrastructure\ConstruSoftTicket.Infrastructure.csproj	Referencia agregada

Paso 5. Crear la entidad del dominio

Este paso ya no se realiza en la terminal, sino en el explorador de archivos de VS Code.

Aspecto	Indicaciones
Carpeta de trabajo	backend/ConstruSoftTicket.Domain
Acción	Crear una carpeta llamada Entities
Archivo a crear	Ticket.cs
Ruta final	backend/ConstruSoftTicket.Domain/Entities/Ticket.cs

Código sugerido

Ticket.cs

backend > ConstruSoftTicket.Domain > Entities > Ticket.cs > { } ConstruSoftTicket.Domain.Entities > Ticket

```
1   namespace ConstruSoftTicket.Domain.Entities;
2
    0 referencias
3   public class Ticket
4   {
        0 referencias
5       public Guid Id { get; set; }
        0 referencias
6       public string Titulo { get; set; } = string.Empty;
        0 referencias
7       public string Descripcion { get; set; } = string.Empty;
        0 referencias
8       public DateTime FechaCreacion { get; set; }
        0 referencias
9       public string Estado { get; set; } = string.Empty;
10  }
```

Resultado esperado

La entidad Ticket representa el objeto principal del dominio en esta etapa inicial.

Paso 6. Crear el caso de uso en Application

Este paso se realiza creando carpetas y archivos dentro de ConstruSoftTicket.Application.

Elemento	Ruta sugerida
Carpeta DTOs	backend/ConstruSoftTicket.Application/DTOs
Carpeta Interfaces	backend/ConstruSoftTicket.Application/Interfaces
Carpeta Services	backend/ConstruSoftTicket.Application/Services

Archivo 1. DTO

Ruta:

backend/ConstruSoftTicket.Application/DTOs/CreateTicketDto.cs

Código:

Ticket.cs | CreateTicketDto.cs

backend > ConstruSoftTicket.Application > DTOs > CreateTicketDto.cs > { } ConstruSoftTicket.Application.DTOs

```
1  namespace ConstruSoftTicket.Application.DTOs;
2
   0 referencias
3  public class CreateTicketDto
4  {
       0 referencias
5      public string Titulo { get; set; } = string.Empty;
       0 referencias
6      public string Descripcion { get; set; } = string.Empty;
7  }
```

Archivo 2. Interfaz

Ruta:

backend/ConstruSoftTicket.Application/Interfaces/ITicketService.cs

Código:

Ticket.cs | CreateTicketDto.cs | ITicketService.cs

backend > ConstruSoftTicket.Application > Interfaces > ITicketService.cs > { } ConstruSoftTicket.Application.Inte

```
1  using ConstruSoftTicket.Application.DTOs;
2
3  namespace ConstruSoftTicket.Application.Interfaces;
4
   0 referencias
5  public interface ITicketService
6  {
       0 referencias
7      void CrearTicket(CreateTicketDto dto);
8  }
```

Archivo 3. Servicio

Ruta:

backend/ConstruSoftTicket.Application/Services/TicketService.cs

Código:

Ticket.cs | CreateTicketDto.cs | ITicketService.cs | TicketService.cs

backend > ConstruSoftTicket.Application > Services > TicketService.cs > { } ConstruSoftTicket.Application.Services

```
using ConstruSoftTicket.Application.DTOs;
using ConstruSoftTicket.Application.Interfaces;

namespace ConstruSoftTicket.Application.Services;

0 referencias
public class TicketService : ITicketService
{
    1 referencia
    public void CrearTicket(CreateTicketDto dto)
    {
        // Implementación inicial de la lógica del caso de uso.
    }
}
```

Paso 7. Crear el controlador en API

Este paso se realiza creando una carpeta o usando la carpeta Controllers ya generada por la plantilla Web API.

Aspecto	Indicaciones
Proyecto	**backend/ConstruSoftTicket.API**
Carpeta	**Controllers**
Archivo	**TicketController.cs**

Código sugerido

TicketController.cs

backend > ConstruSoftTicket.API > Controllers > TicketController.cs > { } ConstruSoftTicket.API.Controllers > Ticket

```
1   using ConstruSoftTicket.Application.DTOs;
2   using ConstruSoftTicket.Application.Interfaces;
3   using Microsoft.AspNetCore.Mvc;
4
5   namespace ConstruSoftTicket.API.Controllers;
6
7   [ApiController]
8   [Route("api/[controller]")]
    1 referencia
9   public class TicketController : ControllerBase
10  {
        2 referencias
11      private readonly ITicketService ticketService;
12
        0 referencias
13      public TicketController(ITicketService ticketService)
14      {
15          this.ticketService = ticketService;
16      }
17
18      [HttpPost]
        0 referencias
19      public IActionResult Crear([FromBody] CreateTicketDto dto)
20      {
21          ticketService.CrearTicket(dto);
22          return Ok(new { mensaje = "Ticket registrado correctamente." });
23      }
24  }
```

Paso 8. Registrar dependencias en Program.cs

Este paso se realiza dentro de:

backend/ConstruSoftTicket.API/Program.cs

Acción

Registrar el servicio para que la API pueda resolver la dependencia.

Código a agregar

TicketController.cs | Program.cs

backend > ConstruSoftTicket.API > Program.cs > ...

```
using ConstruSoftTicket.Application.Interfaces;
using ConstruSoftTicket.Application.Services;

var builder = WebApplication.CreateBuilder(args);

// Add services to the container.
// Learn more about configuring Swagger/OpenAPI at https://aka.ms/aspnetcore/swashbuckle
builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

var app = builder.Build();

// Configure the HTTP request pipeline.
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();

var summaries = new[]
{
    "Freezing", "Bracing", "Chilly", "Cool", "Mild", "Warm", "Balmy", "Hot", "Sweltering", "Scorching"
};
```

Paso 9. Ejecutar la API

La ejecución debe realizarse desde la carpeta del proyecto API.

Aspecto	macOS	Windows 11
Ubicación	cd backend/ConstruSoftTicket.API	cd backend\ConstruSoftTicket.API
Comando de ejecución	dotnet run	dotnet run
Resultado esperado	La API debe iniciar y mostrar la URL local de ejecución.	La API debe iniciar y mostrar la URL local de ejecución.

Verificación

Abrir en navegador la interfaz Swagger que expone la API al ejecutarse en entorno de desarrollo.

Paso 10. Crear el frontend con React y Vite

Este paso debe ejecutarse desde la carpeta raíz ConstruSoftTicket, no desde backend.

Aspecto	macOS	Windows 11
Ubicación previa	Volver a la carpeta raíz del proyecto.	Volver a la carpeta raíz del proyecto
Comando para volver	cd .. si está en backend, o la cantidad necesaria hasta llegar a ConstruSoftTicket	cd .. si está en backend, o la cantidad necesaria hasta llegar a ConstruSoftTicket
Comando de creación	npm create vite@latest frontend	npm create vite@latest frontend
Selección	React	React
Variante	JavaScript	JavaScript
Resultado esperado	Se crea la carpeta frontend	Se crea la carpeta frontend

Paso 11. Instalar dependencias y ejecutar el frontend

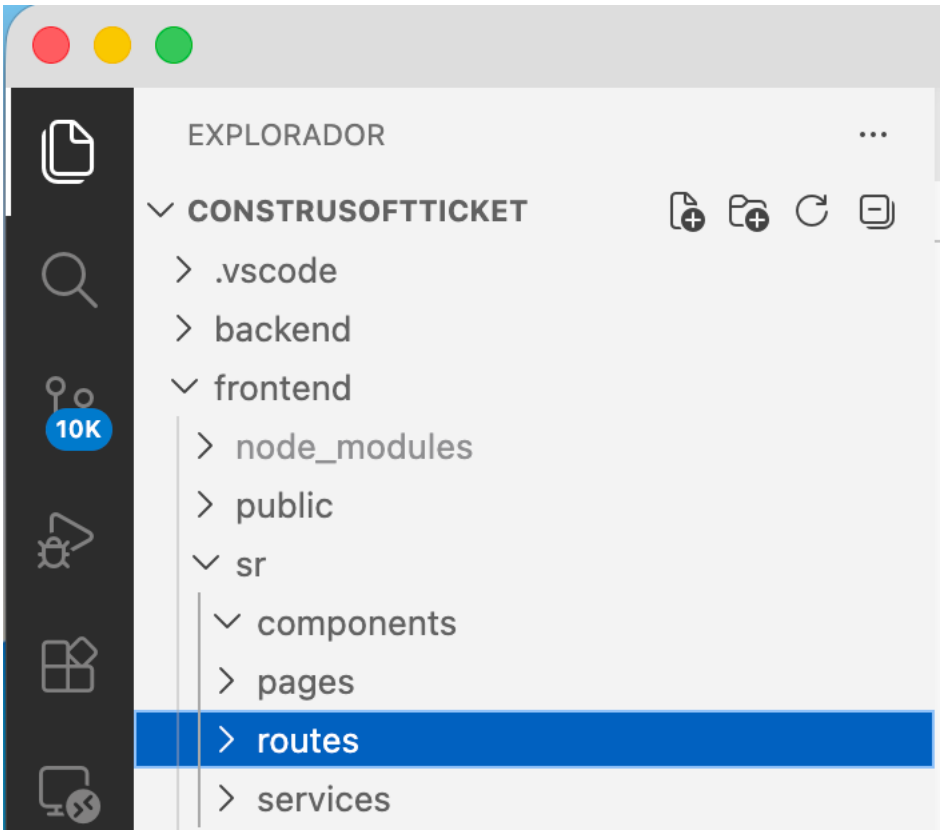
Este paso debe realizarse desde la carpeta frontend.

Aspecto	macOS	Windows 11
Ubicación	cd frontend	cd frontend
Instalar dependencias	npm install	npm install
Ejecutar aplicación	npm run dev	npm run dev
Resultado esperado	Vite muestra la URL local del frontend	Vite muestra la URL local del frontend

Paso 12. Organizar la estructura inicial del frontend

Este paso se realiza desde el explorador de archivos de VS Code.

Carpetas sugeridas dentro de frontend/src



Paso 13. Crear la primera página del sistema

Archivo:

frontend/src/pages/CreateTicket.jsx

CreateTicket.jsx

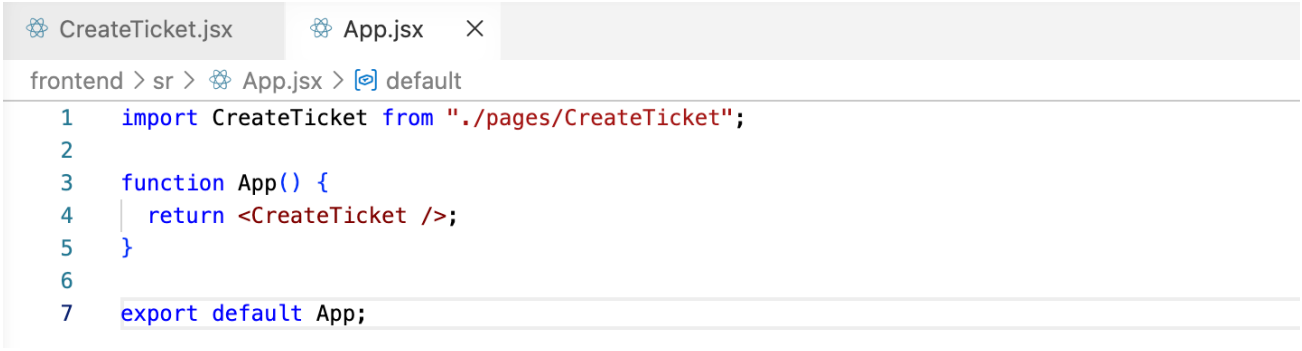
frontend > sr > pages > CreateTicket.jsx > CreateTicket

```
3   export default function CreateTicket() {
17          <div>
18            <label>Título</label>
19            <input
20              type="text"
21              value={titulo}
22              onChange={(event) => setTitulo(event.target.value)}
23            />
24          </div>
25
26          <div>
27            <label>Descripción</label>
28            <textarea
29              value={descripcion}
30              onChange={(event) => setDescripcion(event.target.value)}
31            />
32          </div>
33
34          <button type="submit">Registrar ticket</button>
35        </form>
36      </div>
37    );
38  }
```

Paso 14. Mostrar la página en App.jsx

Archivo:

frontend/src/App.jsx



Resultado esperado

Al ejecutar el frontend, debe mostrarse el formulario de registro del ticket.

Paso 15. Configurar PostgreSQL

En esta semana el objetivo es dejar preparada la base de datos, no necesariamente completar toda la persistencia.

Aspecto	macOS	Windows 11
Herramienta sugerida	pgAdmin o terminal SQL	pgAdmin o terminal SQL
Nombre de base de datos	ConstruSoftTicketDB	ConstruSoftTicketDB
Resultado esperado	Base creada correctamente.	Base creada correctamente.

Script básico sugerido

CREATE DATABASE "ConstruSoftTicketDB";

Paso 16. Validación final de la semana

Al finalizar, el docente debe verificar que el estudiante haya conseguido los siguientes resultados mínimos:

Componente	Evidencia esperada
Estructura raíz	Carpeta ConstruSoftTicket con backend y frontend
Solución .NET	Archivo ConstruSoftTicket.sln
Proyectos backend	Domain, Application, Infrastructure y API

Referencias	Relaciones correctas entre proyectos
Dominio	Entidad Ticket creada
Aplicación	DTO, interfaz y servicio
API	Controlador funcional y servicio registrado
Frontend	Proyecto Vite en ejecución
Interfaz	Formulario inicial de ticket
Base de datos	PostgreSQL preparada

Conclusión

La Semana 03 marca el inicio real del Sistema ConstruSoft Ticket. La prioridad no es desarrollar muchas funcionalidades todavía, sino construir una base técnica correcta que permita integrar progresivamente los módulos de entrada, salida, pruebas y mantenimiento en coherencia con el sílabo del curso.

UPLA

Construcción de software

Semana 04
(Laboratorio)

Aplicación de estándares en el desarrollo del Sistema ConstruSoft Ticket

ASIGNATURA
Construcción de software

ACTIVIDAD 01

“El software es una gran combinación entre arte e ingeniería.”

—Bill Gates

1. **Objetivo**

   Aplicar estándares de construcción de software en un caso práctico mediante la implementación funcional del módulo de registro de tickets en el sistema ConstruSoft Ticket, integrando arquitectura, lógica de negocio y persistencia de datos.

2. **Contexto**

   En la semana anterior, el equipo de desarrollo logró definir la estructura base del sistema ConstruSoft Ticket, estableciendo una arquitectura organizada y alineada a buenas prácticas.

   Sin embargo, el sistema aún no cumple su propósito principal: gestionar información real.

   Actualmente, el registro de tickets solo existe a nivel de interfaz o lógica básica, pero no se almacena en ningún sistema persistente, lo que limita su utilidad.

   Para resolver este problema, el equipo debe avanzar hacia una implementación más completa, integrando:

   - Persistencia de datos
   - Conexión entre capas

- Validación de información
- Flujo completo de registro

Tal como señalan Martin Fowler y Robert C. Martin, la calidad del software no solo depende de su estructura, sino de su correcta implementación funcional.

3. **Problema de la semana**

¿Cómo implementar un módulo funcional de registro de tickets que permita almacenar información en una base de datos, manteniendo una arquitectura limpia y aplicando estándares de construcción de software?

4. **Condiciones y restricciones**

El estudiante debe considerar:

a. Uso del stack definido:
   - C# (.NET)
   - React + Vite
   - PostgreSQL
b. Mantener la arquitectura de capas.
c. Aplicar principios SOLID
d. Implementar persistencia de datos.
e. Garantizar que el flujo funcione de extremo a extremo:
   - Frontend → API → Base de datos

5. **Actividades / Preguntas orientadoras**

El estudiante deberá:

a) ¿Cómo integrará PostgreSQL con el backend del sistema?

b) ¿Qué cambios debe realizar en la capa Infrastructure?
c) ¿Cómo se relaciona la entidad Ticket con la base de datos?
d) ¿Cómo se implementará la persistencia (guardar datos)?
e) ¿Cómo validar que los datos se registran correctamente?
f) ¿Cómo conectar el frontend con el backend?
g) ¿Qué mejoras aplicaría al formulario del sistema?
h) ¿Qué estándares y principios se mantienen en esta etapa?

6. Entregable

El estudiante deberá presentar:

1. Backend con persistencia de datos implementada.
2. Base de datos configurada (PostgreSQL)
3. Registro funcional de tickets (almacenamiento real)
4. Frontend conectado con API
5. Evidencia de funcionamiento (capturas o video)
6. Documento breve justificando:
   - Integración de capas
   - Uso de base de datos
   - Estándares aplicados

7. Reflexión final

- ¿Qué diferencia existe entre un sistema sin persistencia y uno con base de datos?
- ¿Qué retos encontró al integrar múltiples capas del sistema?
- ¿Cómo influye la persistencia en la calidad del software?

Solución (modelo)

Solución modelo: Inicio del desarrollo del sistema ConstruSoft Ticket

1. **Propósito de la actividad**

   Implementar el registro funcional de tickets en el sistema ConstruSoft Ticket, integrando:

   - Persistencia en PostgreSQL;
   - Acceso a datos en la capa Infrastructure;
   - Contratos en Application;
   - Exposición del endpoint en API;
   - Consumo básico desde React.

   Esta semana corrige una decisión que no era compatible con Clean Architecture: usar AppDbContext directamente desde la capa Application. La solución correcta será usar un repositorio definido como interfaz en Application e implementado en Infrastructure.

2. **Antes de comenzar: corrección de estructura**

   2.1 Error que no debe mantenerse

   Si en el archivo:
   backend/ConstruSoftTicket.Application/Services/TicketService.cs

   se colocó algo como:
   using ConstruSoftTicket.Infrastructure.Data;

   eso debe corregirse, porque:

   - Application no debe depender de Infrastructure;
   - DbContext pertenece a Infrastructure;
   - La capa Application solo debe conocer interfaces.

3. **Archivos que deben eliminarse o corregirse**

   3.1 Archivo que debe corregirse

   Ruta:

backend/ConstruSoftTicket.Application/Services/TicketService.cs

Acción:

- No eliminar el archivo
- Sí reemplazar completamente su contenido por la versión correcta que se mostrará más adelante.

3.2 Línea que debe eliminarse si existe

En el archivo:
backend/ConstruSoftTicket.Application/Services/TicketService.cs

si existe esta línea:
using ConstruSoftTicket.Infrastructure.Data;

debe eliminarse.

3.3. Archivos que pueden eliminarse si fueron creados incorrectamente

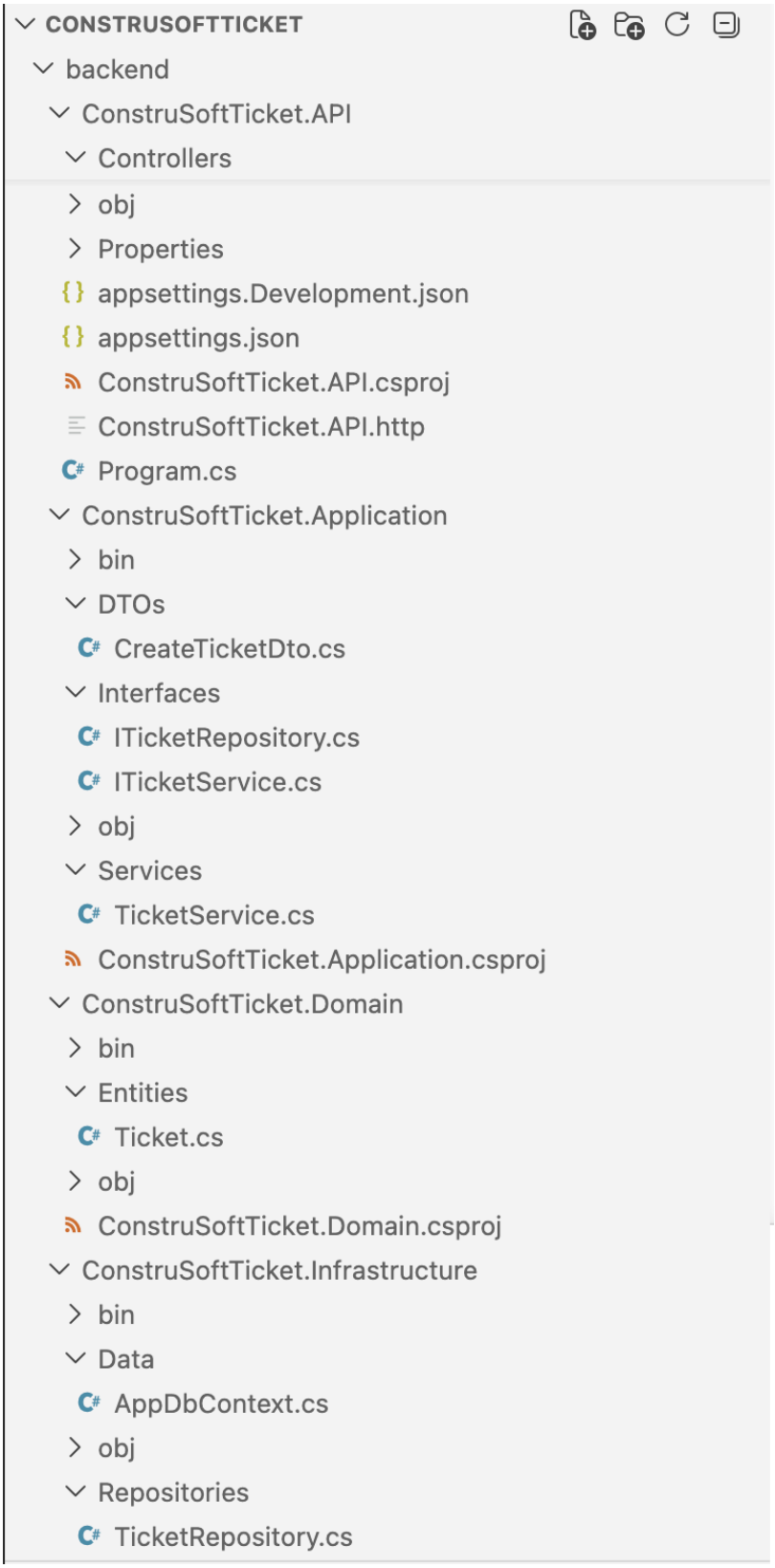
Si durante pruebas previas se creó algún archivo experimental fuera de la estructura correcta, por ejemplo:

- Repositorios dentro de Application con acceso a DbContext;
- Clases temporales duplicadas;
- Servicios que dependan directamente de Infrastructure;

deben eliminarse.

4. **Estructura final esperada del backend**

   Al terminar esta semana, el backend debe quedar así:



5. Desarrollo paso a paso

Paso 1. Verificar paquetes en Infrastructure

Objetivo

Asegurar que Infrastructure tenga las dependencias necesarias para EF Core y PostgreSQL.

Ubicación desde donde se ejecuta

Entrar a:

backend/ConstruSoftTicket.Infrastructure

Comandos

macOS	Windows 11
cd backend/ConstruSoftTicket.Infrastructure	cd backend\ConstruSoftTicket.Infrastructure

Luego ejecutar:

dotnet add package Microsoft.EntityFrameworkCore --version 8.0.6

dotnet add package Npgsql.EntityFrameworkCore.PostgreSQL --version 8.0.4

dotnet add package Microsoft.EntityFrameworkCore.Tools --version 8.0.6

Resultado esperado

El archivo:

backend/ConstruSoftTicket.Infrastructure/ConstruSoftTicket.Infrastructure.csproj

debe contener referencias similares a estas:

```
7
8   <ItemGroup>
9     <PackageReference Include="Microsoft.EntityFrameworkCore" Version="8.0.6" />
10    <PackageReference Include="Microsoft.EntityFrameworkCore.Tools" Version="8.0.6">
11      <IncludeAssets>runtime; build; native; contentfiles; analyzers; buildtransitive</IncludeAssets>
12      <PrivateAssets>all</PrivateAssets>
13    </PackageReference>
14    <PackageReference Include="Npgsql.EntityFrameworkCore.PostgreSQL" Version="8.0.4" />
15  </ItemGroup>
```

Paso 2. Crear o verificar la entidad Ticket

Ruta exacta

backend/ConstruSoftTicket.Domain/Entities/Ticket.cs

Si la carpeta Entities no existe

Crearla manualmente en VS Code.

Contenido final del archivo

Ticket.cs

backend > ConstruSoftTicket.Domain > Entities > Ticket.cs > { } ConstruSoftTicket.Domain.Entitie

```
1   namespace ConstruSoftTicket.Domain.Entities;
2
    5 referencias
3   public class Ticket
4   {
        1 referencia
5       public Guid Id { get; set; }
        1 referencia
6       public string Titulo { get; set; } = string.Empty;
        1 referencia
7       public string Descripcion { get; set; } = string.Empty;
        1 referencia
8       public DateTime FechaCreacion { get; set; }
        1 referencia
9       public string Estado { get; set; } = string.Empty;
10  }
```

Paso 3. Crear o verificar el DTO

Ruta exacta

backend/ConstruSoftTicket.Application/DTOs/CreateTicketDto.cs

Si la carpeta Entities no existe

Crearla manualmente en VS Code.

Contenido final del archivo

CreateTicketDto.cs

backend > ConstruSoftTicket.Application > DTOs > CreateTicketDto.cs > { } ConstruSoftTicket.Ap

```
1   namespace ConstruSoftTicket.Application.DTOs;
2
    3 referencias
3   public class CreateTicketDto
4   {
        1 referencia
5       public string Titulo { get; set; } = string.Empty;
        1 referencia
6       public string Descripcion { get; set; } = string.Empty;
7   }
```

Paso 4. Crear o verificar la interfaz del servicio

Ruta exacta

backend/ConstruSoftTicket.Application/Interfaces/ITicketService.cs

Si la carpeta Interfaces no existe

Crearla manualmente.

Contenido final

ITicketService.cs

backend > ConstruSoftTicket.Application > Interfaces > ITicketService.cs > { } ConstruSoftTicket

```
1   using ConstruSoftTicket.Application.DTOs;
2
3   namespace ConstruSoftTicket.Application.Interfaces;
4
    4 referencias
5   public interface ITicketService
6   {
        2 referencias
7       void CrearTicket(CreateTicketDto dto);
8   }
```

Paso 5. Crear la interfaz del repositorio

Ruta exacta

backend/ConstruSoftTicket.Application/Interfaces/ITicketRepository.cs

Contenido final

ITicketRepository.cs

backend > ConstruSoftTicket.Application > Interfaces > ITicketRepository.cs > { } ConstruSoftTic

```
1   using ConstruSoftTicket.Domain.Entities;
2
3   namespace ConstruSoftTicket.Application.Interfaces;
4
    4 referencias
5   public interface ITicketRepository
6   {
        2 referencias
7       void Add(Ticket ticket);
8   }
```

Resultado esperado

Ahora Application define el contrato del repositorio, pero no conoce cómo se guarda en la base de datos.

Paso 6. Corregir el servicio TicketService

Ruta exacta

backend/ConstruSoftTicket.Application/Services/TicketService.cs

Acción

Reemplazar completamente el contenido por este:

TicketService.cs

backend > ConstruSoftTicket.Application > Services > TicketService.cs > { } ConstruSoftTicket.A

```
using ConstruSoftTicket.Application.DTOs;
using ConstruSoftTicket.Application.Interfaces;
using ConstruSoftTicket.Domain.Entities;

namespace ConstruSoftTicket.Application.Services;

2 referencias
public class TicketService : ITicketService
{
    2 referencias
    private readonly ITicketRepository repository;

    0 referencias
    public TicketService(ITicketRepository repository)
    {
        this.repository = repository;
    }

    2 referencias
    public void CrearTicket(CreateTicketDto dto)
    {
        var ticket = new Ticket
        {
            Id = Guid.NewGuid(),
            Titulo = dto.Titulo,
            Descripcion = dto.Descripcion,
            FechaCreacion = DateTime.UtcNow,
            Estado = "Abierto"
        };

        repository.Add(ticket);
    }
}
```

Importante

En este archivo no debe quedar ninguna referencia a:

using ConstruSoftTicket.Infrastructure.Data;

Paso 7. Crear el DbContext en Infrastructure

Ruta exacta

backend/ConstruSoftTicket.Infrastructure/Data/AppDbContext.cs

Si la carpeta Data no existe

Crearla manualmente.

Contenido final

AppDbContext.cs

backend > ConstruSoftTicket.Infrastructure > Data > AppDbContext.cs > { } ConstruSoftTicket.Infrastru

```
1   using ConstruSoftTicket.Domain.Entities;
2   using Microsoft.EntityFrameworkCore;
3
4   namespace ConstruSoftTicket.Infrastructure.Data;
5
        4 referencias
6   public class AppDbContext : DbContext
7   {
            0 referencias
8       public AppDbContext(DbContextOptions<AppDbContext> options) : base(options)
9       {
10      }
11
            1 referencia
12      public DbSet<Ticket> Tickets => Set<Ticket>();
13  }
```

Paso 8. Crear el repositorio concreto en Infrastructure

Ruta exacta

backend/ConstruSoftTicket.Infrastructure/Repositories/TicketRepository.cs

Si la carpeta Repositories no existe

Crearla manualmente.

Contenido final

TicketRepository.cs

kend > ConstruSoftTicket.Infrastructure > Repositories > TicketRepository.cs > { } Con

```
1   using ConstruSoftTicket.Application.Interfaces;
2   using ConstruSoftTicket.Domain.Entities;
3   using ConstruSoftTicket.Infrastructure.Data;
4
5   namespace ConstruSoftTicket.Infrastructure.Repositories;
6
        2 referencias
7   public class TicketRepository : ITicketRepository
8   {
            3 referencias
9       private readonly AppDbContext context;
10
            0 referencias
11      public TicketRepository(AppDbContext context)
12      {
13          this.context = context;
14      }
15
            2 referencias
16      public void Add(Ticket ticket)
17      {
18          context.Tickets.Add(ticket);
19          context.SaveChanges();
20      }
21  }
```

Paso 9. Verificar referencias entre proyectos

Objetivo

Asegurar que los proyectos tengan las referencias necesarias para compilar.

Ubicación desde donde se ejecuta

Entrar a:

backend

Comandos

macOS	Windows 11
cd backend	cd backend

Luego ejecutar:

dotnet add ConstruSoftTicket.Application/ConstruSoftTicket.Application.csproj reference ConstruSoftTicket.Domain/ConstruSoftTicket.Domain.csproj

dotnet add ConstruSoftTicket.Infrastructure/ConstruSoftTicket.Infrastructure.csproj reference ConstruSoftTicket.Application/ConstruSoftTicket.Application.csproj

dotnet add ConstruSoftTicket.Infrastructure/ConstruSoftTicket.Infrastructure.csproj reference ConstruSoftTicket.Domain/ConstruSoftTicket.Domain.csproj

dotnet add ConstruSoftTicket.API/ConstruSoftTicket.API.csproj reference ConstruSoftTicket.Application/ConstruSoftTicket.Application.csproj

dotnet add ConstruSoftTicket.API/ConstruSoftTicket.API.csproj reference ConstruSoftTicket.Infrastructure/ConstruSoftTicket.Infrastructure.csproj

Resultado esperado

- Application referencia a Domain
- Infrastructure referencia a Application y Domain
- API referencia a Application e Infrastructure

Paso 10. Configurar la cadena de conexión

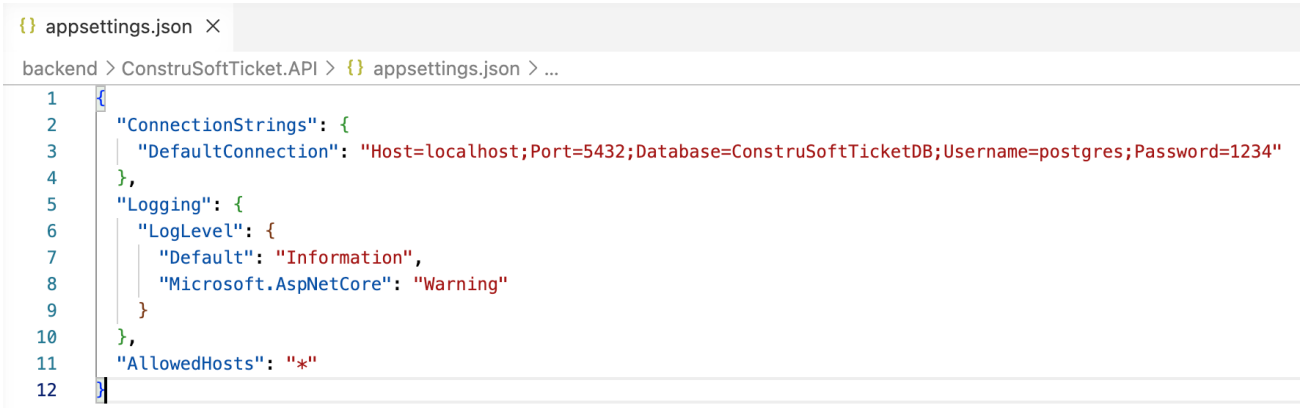
Ruta exacta

backend/ConstruSoftTicket.API/appsettings.json

Acción

Agregar o ajustar la sección ConnectionStrings.

Contenido sugerido



Importante

La contraseña 1234 es solo un ejemplo. Debe reemplazarse por la contraseña real configurada en PostgreSQL.

Paso 11. Corregir Program.cs

Ruta exacta

backend/ConstruSoftTicket.API/Program.cs

Acción

Reemplazar completamente el contenido del archivo por esta versión, para evitar inconsistencias con cambios previos:

Program.cs

backend > ConstruSoftTicket.API > Program.cs

```
using ConstruSoftTicket.Application.Interfaces;
using ConstruSoftTicket.Application.Services;
using ConstruSoftTicket.Infrastructure.Data;
using ConstruSoftTicket.Infrastructure.Repositories;
using Microsoft.EntityFrameworkCore;

var builder = WebApplication.CreateBuilder(args);

// Add services to the container.
builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseNpgsql(builder.Configuration.GetConnectionString("DefaultConnection")));

builder.Services.AddScoped<ITicketRepository, TicketRepository>();
builder.Services.AddScoped<ITicketService, TicketService>();

builder.Services.AddCors(options =>
{
    options.AddPolicy("AllowFrontend", policy =>
    {
        policy.AllowAnyOrigin()
              .AllowAnyHeader()
              .AllowAnyMethod();
    });
});

var app = builder.Build();
// Configure the HTTP request pipeline.
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseCors("AllowFrontend");
app.UseHttpsRedirection();
app.UseAuthorization();
app.MapControllers();
app.Run();
```

Paso 12. Crear o verificar el controlador

Ruta exacta

backend/ConstruSoftTicket.API/Controllers/TicketController.cs

Si la carpeta Controllers no existe

Crearla manualmente.

Contenido final

TicketController.cs

backend > ConstruSoftTicket.API > Controllers > TicketController.cs > { } ConstruSoftTicket.A

```
1   using ConstruSoftTicket.Application.DTOs;
2   using ConstruSoftTicket.Application.Interfaces;
3   using Microsoft.AspNetCore.Mvc;
4
5   namespace ConstruSoftTicket.API.Controllers;
6
7   [ApiController]
8   [Route("api/[controller]")]
    1 referencia
9   public class TicketController : ControllerBase
10  {
        2 referencias
11      private readonly ITicketService ticketService;
12
        0 referencias
13      public TicketController(ITicketService ticketService)
14      {
15          this.ticketService = ticketService;
16      }
17
18      [HttpPost]
        0 referencias
19      public IActionResult Crear([FromBody] CreateTicketDto dto)
20      {
21          ticketService.CrearTicket(dto);
22          return Ok(new { mensaje = "Ticket registrado correctamente." });
23      }
24  }
```

Paso 13. Verificar si PostgreSQL está listo

Objetivo

Confirmar que existe la base de datos ConstruSoftTicketDB.

Opción terminal

macOS	Windows 11
psql postgres o psql -U postgres	psql -U postgres

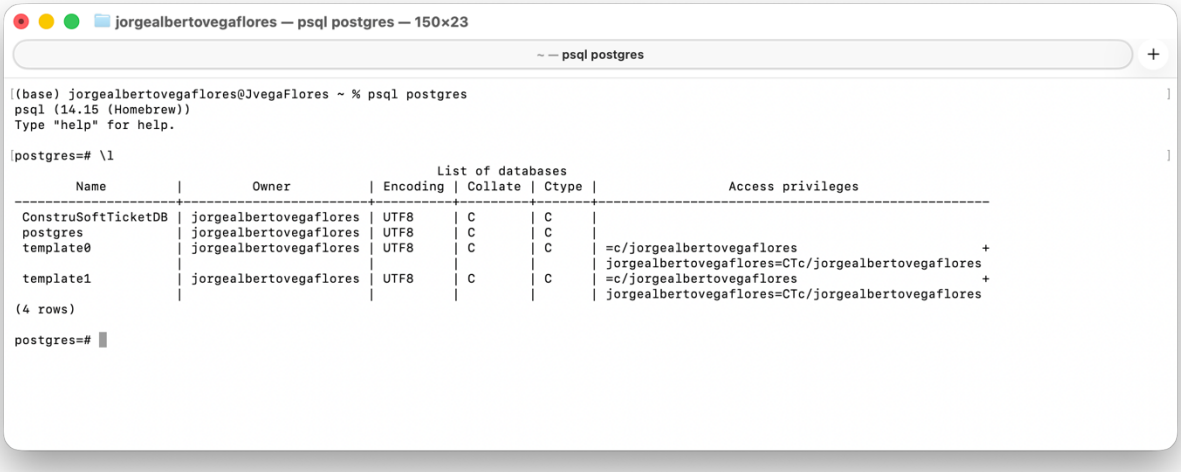
Luego ejecutar:

\l

Resultado esperado

Debe aparecer la base de datos:

ConstruSoftTicketDB



Si no existe

Crear con:

CREATE DATABASE "ConstruSoftTicketDB";

Salir con:

\q

Paso 14. Crear la migración inicial

Ubicación desde donde se ejecuta

Entrar a:

backend/ConstruSoftTicket.API

Comandos

macOS	Windows 11
cd backend/ConstruSoftTicket.API	cd backend\ConstruSoftTicket.API

Luego ejecutar:
dotnet ef migrations add InitialCreate --project ../ConstruSoftTicket.Infrastructure --startup-project .

En caso de error de referencia instalar en backend/ConstruSoftTicket.API

Comando
dotnet add package Microsoft.EntityFrameworkCore.Design --version 8.0.6

Resultado esperado
Se crea una carpeta Migrations dentro del proyecto Infrastructure.

Paso 15. Aplicar la migración a la base de datos.
Desde la misma carpeta
backend/ConstruSoftTicket.API

Comando
dotnet ef database update --project ../ConstruSoftTicket.Infrastructure --startup-project .

Resultado esperado
La tabla Tickets queda creada en ConstruSoftTicketDB.

Paso 16. Ejecutar la API
Ubicación exacta
backend/ConstruSoftTicket.API

Comandos

macOS	Windows 11
dotnet run	dotnet run

Resultado esperado
La terminal mostrará una URL similar a:

```
(base) jorgealbertovegaflores@JvegaFlores ConstruSoftTicket.API % dotnet run
Compilando...
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: http://localhost:5153
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]
      Content root path: /Users/jorgealbertovegaflores/Documents/ConstruSoftTicket/backend/ConstruSoftTicket.API
warn: Microsoft.AspNetCore.HttpsPolicy.HttpsRedirectionMiddleware[3]
      Failed to determine the https port for redirect.
```

Paso 17. Probar en Swagger

Acción

Abrir en navegador:

https://localhost:XXXX/swagger

Endpoint a probar

POST /api/ticket

JSON de prueba

```
{
  "titulo": "PC no enciende",
  "descripcion": "Equipo del laboratorio 3 no responde al encendido."
}
```

Resultado esperado

Respuesta:

```
{
  "mensaje": "Ticket registrado correctamente."
}
```

Paso 18. Verificar en PostgreSQL

Comando de acceso

macOS	Windows 11
psql -U postgres -d ConstruSoftTicketDB	psql -U postgres -d ConstruSoftTicketDB

Consulta

SELECT * FROM "Tickets";

Resultado esperado

Debe aparecer al menos un ticket insertado.

Paso 19. Conectar el frontend con el backend

Ruta exacta

frontend/src/pages/CreateTicket.jsx

Acción

Reemplazar completamente el contenido por esta versión:

CreateTicket.jsx

frontend > src > pages > CreateTicket.jsx > CreateTicket > handleSubmit

```jsx
import { useState } from "react";

export default function CreateTicket() {
  const [titulo, setTitulo] = useState("");
  const [descripcion, setDescripcion] = useState("");

  const handleSubmit = async (event) => {
    event.preventDefault();

    const response = await fetch("https://localhost:7208/api/ticket", {
      method: "POST",
      headers: {
        "Content-Type": "application/json"
      },
      body: JSON.stringify({
        titulo,
        descripcion
      })
    });

    if (response.ok) {
      alert("Ticket registrado correctamente");
      setTitulo("");
      setDescripcion("");
    } else {
      alert("Error al registrar el ticket");
    }
  };

```

```
  return (
    <div>
      <h1>Registro de Ticket</h1>

      <form onSubmit={handleSubmit}>
        <div>
          <label>Título</label>
          <input
            type="text"
            value={titulo}
            onChange={(event) => setTitulo(event.target.value)}
          />
        </div>

        <div>
          <label>Descripción</label>
          <textarea
            value={descripcion}
            onChange={(event) => setDescripcion(event.target.value)}
          />
        </div>

        <button type="submit">Registrar ticket</button>
      </form>
    </div>
  );
}
```

Importante

Si el puerto del backend no es 7208, debe reemplazarse por el puerto real mostrado por dotnet run.

Paso 20. Verificar App.jsx

Ruta exacta

frontend/src/App.jsx

Contenido esperado

App.jsx

frontend > src > App.jsx > default

```jsx
import CreateTicket from "./pages/CreateTicket";

function App() {
  return <CreateTicket />;
}

export default App;
```

Paso 21. Ejecutar el frontend

Ubicación exacta

frontend

Comandos

macOS	Windows 11
cd frontend	**cd frontend**
npm install	**npm install**
npm run dev	**npm run dev**

Resultado esperado

El navegador mostrará la pantalla de registro de tickets.

6. **Archivos que deben quedar finalmente en el proyecto**

Backend

Domain

- backend/ConstruSoftTicket.Domain/Entities/Ticket.cs

Application

- backend/ConstruSoftTicket.Application/DTOs/CreateTicketDto.cs
- backend/ConstruSoftTicket.Application/Interfaces/ITicketRepository.cs
- backend/ConstruSoftTicket.Application/Interfaces/ITicketService.cs

- backend/ConstruSoftTicket.Application/Services/TicketService.cs

Infrastructure

- backend/ConstruSoftTicket.Infrastructure/Data/AppDbContext.cs
- backend/ConstruSoftTicket.Infrastructure/Repositories/TicketRepository.cs
- carpeta Migrations generada por EF Core
- archivo .csproj con paquetes correctos

API

- backend/ConstruSoftTicket.API/Controllers/TicketController.cs
- backend/ConstruSoftTicket.API/Program.cs
- backend/ConstruSoftTicket.API/appsettings.json

Frontend

- frontend/src/pages/CreateTicket.jsx
- frontend/src/App.jsx

7. Archivos que no deben quedar con contenido incorrecto

No debe quedar en TicketService.cs

- Ninguna referencia a ConstruSoftTicket.Infrastructure.Data
- Ninguna dependencia directa a AppDbContext

No debe haber repositorios dentro de Application

que implementen acceso real a la base de datos.

8. Criterios de verificación docente

Elemento	Verificación
Arquitectura	Application no depende de Infrastructure
Persistencia	Infrastructure contiene AppDbContext y TicketRepository
API	expone POST /api/ticket
Base de datos	tabla Tickets creada
Frontend	envía datos al backend
Flujo	datos se guardan realmente en PostgreSQL

UPLA

Construcción de software

Semana 05
(Laboratorio)

Identificar los principios de construcción de módulos de entrada de datos en software.

ASIGNATURA
Construcción de software

ACTIVIDAD 01

“Un sistema es tan bueno como los datos que recibe.”

—Tom DeMarco

1. **Objetivo**

   Analizar y mejorar el módulo de entrada de datos del sistema ConstruSoft Ticket, identificando los principios de construcción de formularios de software y proponiendo mejoras que garanticen calidad, consistencia y usabilidad de la información capturada.

2. **Contexto**

   En la semana anterior, el equipo logró implementar un flujo funcional completo del sistema:

   - El usuario registra un ticket desde el frontend.
   - La API procesa la solicitud.
   - La información se almacena en PostgreSQL.

   Sin embargo, el sistema aún presenta una debilidad importante:
   No existe control sobre la calidad de los datos ingresados.

   Actualmente, el formulario permite registrar información sin restricciones, lo que puede generar problemas como:

   - Registros incompletos
   - Datos inconsistentes
   - Dificultades en la gestión de incidencias

Tal como señalan Roger S. Pressman e Ian Sommerville, los módulos de entrada de datos deben diseñarse cuidadosamente para garantizar la integridad de la información y la correcta interacción con el usuario.

3. Problema de la semana

¿Cómo mejorar el módulo de entrada de datos del sistema ConstruSoft Ticket para garantizar que la información registrada sea válida, consistente y útil para la gestión de incidencias?

4. Condiciones y restricciones

El estudiante debe considerar:

- Uso del sistema ya implementado en la Semana 04
- Mantener la arquitectura de capas (Clean Architecture).
- No modificar la estructura general del sistema.
- No implementar aún validaciones complejas (esto se verá en la siguiente semana).
- Analizar el comportamiento actual del formulario.
- Proponer mejoras fundamentadas

5. Actividades / Preguntas orientadoras

El estudiante deberá desarrollar lo siguiente:

1. Análisis del formulario actual
   - ¿Qué campos tiene el formulario de registro de tickets?
   - ¿Todos los campos son necesarios?
   - ¿Existe control sobre datos vacíos?

2. Identificación de problemas
    - ¿Qué ocurre si el usuario envía el formulario sin completar datos?
    - ¿Se pueden ingresar datos inválidos o irrelevantes?
    - ¿Qué impacto tendría esto en el sistema?

3. Evaluación de calidad de datos
    - ¿Los datos registrados son útiles para resolver incidencias?
    - ¿Qué información adicional podría mejorar el sistema?

4. Propuesta de mejora del módulo de entrada

    El estudiante debe proponer mejoras como:
    - Definición de campos obligatorios
    - Mejora en la estructura del formulario
    - Organización de la interfaz
    - Claridad en etiquetas y campos
    - Sugerencias de validación futura

5. Análisis desde principios de construcción de software
    - ¿Qué principios de calidad se están aplicando o incumpliendo?
    - ¿Cómo influye el diseño del formulario en la calidad del sistema?

6. Entregable

El estudiante deberá presentar:

1. Documento breve que contenga:
    - Análisis del formulario actual.
    - Identificación de problemas.

- Propuesta de mejora.

2. Capturas del sistema en ejecución:
   - Formulario actual.
   - Ejemplo de registro de ticket.

3. Prototipo mejorado del formulario
   - Puede ser en código o diseño.

7. **Reflexión final**

   El estudiante deberá responder:

   - ¿Por qué es importante controlar la entrada de datos en un sistema?
   - ¿Qué consecuencias puede tener un mal diseño de formularios?
   - ¿Cómo influye este módulo en la calidad global del software?

Solución (modelo)

Principios de construcción de módulos de entrada de datos en el Sistema ConstruSoft Ticket

1. **Propósito de la actividad**

   Orientar al docente en el desarrollo de la Semana 05, correspondiente al inicio de la Unidad II: Construcción de módulos de entrada de datos, cuyo desempeño se centra en identificar los principios de construcción de módulos de entrada de datos en software.

   En esta semana, el estudiante no debe limitarse a “hacer funcionar” el formulario, sino que debe analizar si el módulo de entrada permite capturar datos claros, consistentes, útiles y preparados para su validación posterior.

2. **Punto de partida de la semana**

   Al iniciar la Semana 05, el proyecto ConstruSoft Ticket debe contar con lo desarrollado en la Semana 04:

Componente	Estado esperado
Backend	**API funcional en ASP.NET Core**
Frontend	**React + Vite ejecutándose**
Base de datos	**PostgreSQL con tabla Tickets**
Flujo	**React registra tickets en PostgreSQL.**
Endpoint	**POST /api/ticket funcionando**
Arquitectura	**Clean Architecture aplicada**

3. **Archivos que deben eliminarse o corregirse**

   Aunque el sistema ya registra tickets, el formulario todavía es básico. Esto genera una pregunta técnica importante:

   ¿El módulo de entrada está capturando datos de manera clara, ordenada y útil para el sistema?

   La solución modelo consiste en mejorar el formulario de registro de tickets, aplicando principios iniciales de construcción de módulos de entrada:

- Claridad de campos;
- Obligatoriedad básica;
- Mensajes comprensibles;
- Separación de responsabilidades en React;
- Preparación para validaciones más completas en la Semana 06.

4. Estructura final esperada del backend

4.1. Alcance técnico de la Semana 05

En esta semana se trabajará principalmente sobre el frontend, sin modificar todavía la estructura de la base de datos.

Se mantendrán los campos actuales:

- titulo
- descripcion

Y se mejorará:

- La organización del formulario;
- La experiencia de entrada;
- La validación básica en el cliente;
- El manejo de mensajes;
- La separación del consumo de API en un archivo de servicio.

5. Desarrollo paso a paso

5.1. Archivos que se conservan

Archivo	Acción
backend/ConstruSoftTicket.Domain/Entities/Ticket.cs	No modificar
backend/ConstruSoftTicket.Application/DTOs/CreateTicketDto.cs	No modificar
backend/ConstruSoftTicket.Application/Services/TicketService.cs	No modificar
backend/ConstruSoftTicket.Infrastructure/Data/AppDbContext.cs	No modificar
backend/ConstruSoftTicket.Infrastructure/Repositories/TicketRepository.cs	No modificar
backend/ConstruSoftTicket.API/Controllers/TicketController.cs	No modificar
backend/ConstruSoftTicket.API/Program.cs	No modificar si ya funciona.

backend/ConstruSoftTicket.API/appsettings.json	No modificar si ya conecta con PostgreSQL.

5.2. Archivos que se modificarán

Archivo	Acción
frontend/src/pages/CreateTicket.jsx	Reemplazar por una versión mejorada.
frontend/src/App.jsx	Verificar que renderice CreateTicket.

5.3. Archivos que se crearán

Archivo	Propósito
frontend/src/services/ticketService.js	**Centralizar la comunicación con la API.**
frontend/src/components/FormMessage.jsx	**Mostrar mensajes de éxito o error.**
frontend/src/components/InputField.jsx	**Reutilizar estructura de campos de entrada.**

6. Desarrollo paso a paso

Paso 1. Levantar el backend

Ruta exacta

/Users/jorgealbertovegaflores/Documents/ConstruSoftTicket/backend/ConstruSoftTicket.API

Comandos

macOS	Windows 11
cd /Users/jorgealbertovegaflores/Documents/ConstruSoftTicket/backend/ConstruSoftTicket.API dotnet run	cd C:\Ruta\ConstruSoftTicket\backend\ConstruSoftTicket.API dotnet run

Resultado esperado

Debe mostrarse una URL similar a:

http://localhost:5153

También debe abrir Swagger en:

http://localhost:5153/swagger/index.html

Paso 2. Levantar el frontend

Abrir una segunda terminal.

Ruta exacta en macOS

/Users/jorgealbertovegaflores/Documents/ConstruSoftTicket/frontend

macOS

cd /Users/jorgealbertovegaflores/Documents/ConstruSoftTicket/frontend

npm run dev

Windows 11

cd C:\Ruta\ConstruSoftTicket\frontend

npm run dev

Resultado esperado

Debe mostrarse:

http://localhost:5173/

7. **Crear servicio para consumir la API.**

   Paso 3. Crear archivo ticketService.js

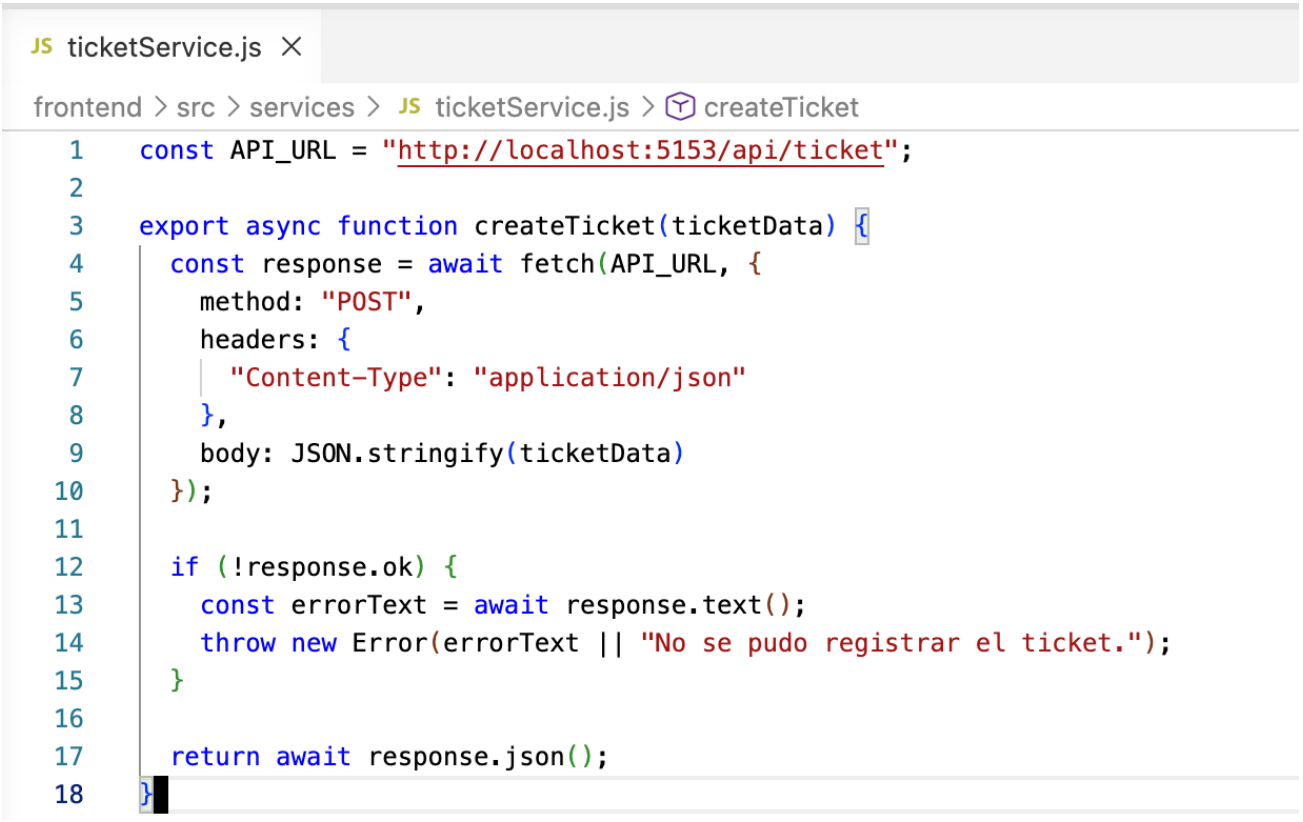
   Ruta exacta

   frontend/src/services/ticketService.js

   Si la carpeta services no existe, crearla dentro de:

   frontend/src/

   Código



Explicación

Este archivo separa la lógica de comunicación con la API del componente visual. De esta forma, el formulario no queda sobrecargado con detalles técnicos del endpoint.

8. **Crear componente reutilizable para mensajes**

Paso 4. Crear FormMessage.jsx

Ruta exacta

frontend/src/components/FormMessage.jsx

Si la carpeta components no existe, crearla dentro de:

frontend/src/

Código

FormMessage.jsx

frontend > src > components > FormMessage.jsx > FormMessage

```jsx
export default function FormMessage({ type, message }) {
  if (!message) {
    return null;
  }

  return (
    <p>
      {type === "success" ? "✅ " : "⚠️ "}
      {message}
    </p>
  );
}
```

Explicación

Este componente permite mostrar mensajes de éxito o error sin mezclar esa lógica directamente dentro del formulario.

9. **Crear componente reutilizable para campos**

   Paso 5. Crear InputField.jsx

   Ruta exacta

   frontend/src/components/InputField.jsx

   Código

InputField.jsx

frontend > src > components > InputField.jsx > InputField

```jsx
export default function InputField({
  label,
  name,
  value,
  onChange,
  placeholder,
  type = "text",
  required = false
}) {
  return (
    <div>
      <label htmlFor={name}>{label}</label>

      <input
        id={name}
        name={name}
        type={type}
        value={value}
        placeholder={placeholder}
        required={required}
        onChange={onChange}
      />
    </div>
  );
}
```

Explicación

Este componente permite aplicar el principio de reutilización. Aunque por ahora solo se use para el título, prepara el proyecto para formularios más completos en semanas posteriores.

10. Mejorar el formulario de registro de tickets

Paso 6. Modificar CreateTicket.jsx

Ruta exacta

frontend/src/pages/CreateTicket.jsx

Acción

Reemplazar completamente el contenido actual por el siguiente:

CreateTicket.jsx

frontend > src > pages > CreateTicket.jsx > CreateTicket

```jsx
import { useState } from "react";
import InputField from "../components/InputField";
import FormMessage from "../components/FormMessage";
import { createTicket } from "../services/ticketService";

export default function CreateTicket() {
  const [formData, setFormData] = useState({
    titulo: "",
    descripcion: ""
  });

  const [message, setMessage] = useState("");
  const [messageType, setMessageType] = useState("");
  const [isSubmitting, setIsSubmitting] = useState(false);

  const handleChange = (event) => {
    const { name, value } = event.target;

    setFormData({
      ...formData,
      [name]: value
    });
  };

  const isFormValid = () => {
    return (
      formData.titulo.trim().length > 0 &&
      formData.descripcion.trim().length > 0
    );
  };

```

```
32  const handleSubmit = async (event) => {
33    event.preventDefault();
34
35    if (!isFormValid()) {
36      setMessageType("error");
37      setMessage("Debe completar el título y la descripción del ticket.");
38      return;
39    }
40
41    try {
42      setIsSubmitting(true);
43      setMessage("");
44      setMessageType("");
45
46      const result = await createTicket({
47        titulo: formData.titulo.trim(),
48        descripcion: formData.descripcion.trim()
49      });
50
51      setMessageType("success");
52      setMessage(result.mensaje || "Ticket registrado correctamente.");
53
54      setFormData({
55        titulo: "",
56        descripcion: ""
57      });
58    } catch (error) {
59      setMessageType("error");
60      setMessage("No se pudo registrar el ticket. Revise la conexión con la API.");
61      console.error("Error al registrar ticket:", error);
62    } finally {
63      setIsSubmitting(false);
64    }
65  };
66
```

```
  return (
    <main>
      <h1>Registro de Ticket</h1>

      <p>
        Complete la información básica de la incidencia técnica. Los datos
        ingresados deben ser claros para facilitar su atención posterior.
      </p>

      <form onSubmit={handleSubmit}>
        <InputField
          label="Título del ticket"
          name="titulo"
          value={formData.titulo}
          placeholder="Ejemplo: PC no enciende"
          required={true}
          onChange={handleChange}
        />

        <div>
          <label htmlFor="descripcion">Descripción de la incidencia</label>

          <textarea
            id="descripcion"
            name="descripcion"
            value={formData.descripcion}
            placeholder="Describa brevemente el problema encontrado"
            required
            rows="5"
            onChange={handleChange}
          />
        </div>

        <button type="submit" disabled={isSubmitting}>
          {isSubmitting ? "Registrando..." : "Registrar ticket"}
        </button>
      </form>

      <FormMessage type={messageType} message={message} />
    </main>
  );
}
```

11. Verificar App.jsx

Paso 7. Revisar archivo principal.

Ruta exacta

frontend/src/App.jsx

Contenido esperado

App.jsx

frontend > src > App.jsx > default

```jsx
import CreateTicket from "./pages/CreateTicket";

function App() {
  return <CreateTicket />;
}

export default App;
```

12. **Probar el formulario mejorado**

Paso 8. Prueba 1: Enviar formulario vacío

Acción

Abrir:

http://localhost:5173/

Intentar registrar sin llenar campos.

Resultado esperado

El sistema debe impedir el envío y mostrar un mensaje indicando que se deben completar los campos.

Paso 9. Prueba 2: Registrar ticket válido

Datos sugeridos

Título: Teclado no responde.

Descripción: El teclado del equipo 04 del laboratorio no permite ingresar texto.

Resultado esperado

Debe mostrarse:

Ticket registrado correctamente.

Y el registro debe guardarse en PostgreSQL.

13. Verificar en PostgreSQL.

Paso 10. Entrar a la base de datos

macOS

psql -d ConstruSoftTicketDB

Si se requiere usuario:

psql -U TU_USUARIO -d ConstruSoftTicketDB

Windows 11

psql -U TU_USUARIO -d ConstruSoftTicketDB

Paso 11. Consultar registros

SELECT "Titulo", "Descripcion", "Estado" FROM "Tickets";

Resultado esperado

Debe aparecer el ticket registrado desde React.

14. Solución modelo de las preguntas orientadoras

1. ¿Qué campos tiene el formulario actual?

   El formulario contiene dos campos principales: titulo y descripcion. Estos permiten registrar la información mínima necesaria para identificar una incidencia técnica.

2. ¿Todos los campos son necesarios?

   Sí. El título permite identificar rápidamente el problema, mientras que la descripción proporciona el detalle mínimo para su atención. Sin estos dos campos, el ticket pierde utilidad operativa.

3. ¿Existe control sobre datos vacíos?

   En la versión inicial no existía un control suficiente. En la solución modelo se incorporó una validación básica mediante:

   - Propiedad requerida;
   - Función isFormValid();
   - Limpieza con trim().

4. ¿Qué ocurre si se envía el formulario sin completar datos?

   El sistema debe impedir el envío y mostrar un mensaje comprensible al usuario. Esto reduce registros vacíos o de baja calidad.

5. ¿Qué impacto tendría registrar datos incompletos?

   Los datos incompletos dificultan la atención de incidencias, generan reprocesos y afectan la confiabilidad del sistema.

6. ¿Qué principios se aplican en esta mejora?

   Se aplican principalmente:

Principio	Aplicación
Separación de responsabilidades	**ticketService.js gestiona comunicación con API**
Reutilización	**InputField.jsx puede usarse en otros formularios.**
Claridad	**Etiquetas y mensajes orientan al usuario.**
Mantenibilidad	**El formulario queda más ordenado.**
Calidad de entrada	**Se evitan datos vacíos.**

15. Errores frecuentes y solución

Error	Causa probable	Solución
Failed to fetch	URL incorrecta o API apagada	Verificar que backend esté en http://localhost:5153
Error SSL	Se usó https en lugar de http.	Cambiar a http://localhost:5153/api/ticket
CORS error	Falta configuración CORS en Program.cs	Verificar app.UseCors("AllowFrontend")
No se guarda en BD	API no recibió datos.	Probar primero desde Swagger.
No aparece mensaje.	FormMessage no está importado.	Revisar ruta ../components/FormMessage
Pantalla en blanco.	Error de importación	Revisar consola del navegador.

16. Criterios de evaluación docente

Criterio	Indicador observable
Análisis del módulo de entrada	El estudiante identifica debilidades del formulario.
Calidad de entrada	Evita registros vacíos.
Organización del frontend	Separa página, componentes y servicios.
Funcionamiento	El ticket se registra correctamente.
Consistencia técnica	Mantiene comunicación con la API.
Reflexión	Explica por qué la entrada de datos afecta la calidad del software.

Recuerda:

“Un módulo de entrada no es solo un formulario. Es el punto donde el sistema empieza a construir la calidad de sus datos. Si la entrada es deficiente, todo el sistema hereda ese problema.”

UPLA

Construcción de software

Semana 06
(Laboratorio)

Aplicar técnicas de validación y captura de datos.

ASIGNATURA
Construcción de software

ACTIVIDAD 01

“La calidad nunca es un accidente; siempre es el resultado de un esfuerzo inteligente.”
— John Ruskin

1. **Objetivo**

   Aplicar técnicas de validación y captura de datos en el módulo de registro de tickets del sistema ConstruSoft Ticket, incorporando controles que permitan garantizar integridad, consistencia y confiabilidad de la información ingresada.

2. **Contexto**

   En la semana anterior, el equipo logró reorganizar el módulo de entrada de datos del sistema:

   - Separó componentes del frontend;
   - Mejoró la estructura del formulario;
   - Incorporó validaciones básicas;
   - Centralizó el consumo de la API.

   Sin embargo, el sistema todavía presenta limitaciones importantes relacionadas con la validación de datos.

   Actualmente:

   - No se controla la longitud mínima de los campos;
   - El backend acepta cualquier contenido recibido;
   - No existen mensajes específicos de validación;
   - El sistema no diferencia distintos tipos de error.

Esto representa un riesgo para la calidad de la información almacenada y para la estabilidad futura del sistema.

Tal como señalan Roger S. Pressman e Ian Sommerville, la validación de datos es una parte esencial de la construcción de software, ya que protege la integridad del sistema y mejora la experiencia del usuario.

3. **Problema de la semana**

   ¿Cómo aplicar técnicas de validación y captura de datos que permitan asegurar que la información registrada en el sistema ConstruSoft Ticket sea válida, consistente y confiable?

4. **Condiciones y restricciones**

   El estudiante debe considerar:

   - Mantener la arquitectura limpia implementada;
   - No romper el flujo actual: React → API → PostgreSQL;
   - Validar tanto en frontend como en backend;
   - Utilizar mensajes claros para el usuario;
   - Evitar duplicar lógica innecesariamente;
   - Mantener separación de responsabilidades.

5. **Actividades / Preguntas orientadoras**

   El estudiante deberá desarrollar lo siguiente:

   1. Análisis de riesgos de entrada de datos
      - ¿Qué problemas puede generar un título demasiado corto?
      - ¿Qué ocurre si la descripción contiene información irrelevante?

- ¿Qué riesgos existen si el backend no valida datos?

2. Identificación de técnicas de validación
   - ¿Qué validaciones deberían aplicarse al título?
   - ¿Qué validaciones deberían aplicarse a la descripción?
   - ¿Qué diferencias existen entre validar en frontend y backend?

3. Mejora del módulo de captura

   El estudiante debe analizar cómo incorporar:
   - Obligatoriedad de campos;
   - Validación de longitud mínima;
   - Validación de longitud máxima;
   - Mensajes de error claros;
   - Control del estado del formulario;
   - Prevención de registros inválidos.

4. Análisis de calidad de software
   - ¿Cómo contribuyen las validaciones a la calidad del sistema?
   - ¿Qué principios de ingeniería de software se están aplicando?
   - ¿Cómo influye la validación en la experiencia del usuario?

5. Reflexión técnica
   - ¿Es suficiente validar solo en frontend?
   - ¿Qué ocurre si un usuario evita las validaciones visuales?
   - ¿Por qué el backend también debe proteger el sistema?

6. **Entregable**

   El estudiante deberá presentar:

1. Sistema actualizado con validaciones implementadas.
2. Capturas de:
    - Validaciones en frontend;
    - Mensajes de error;
    - Registros exitosos;
    - Pruebas de errores.
3. Documento breve que incluya:
    - Técnicas de validación aplicadas;
    - Explicación de controles implementados;
    - Reflexión sobre calidad de datos.

7. Reflexión final

El estudiante deberá responder:

- ¿Qué diferencia existe entre un formulario simple y uno validado correctamente?
- ¿Cómo ayudan las validaciones a proteger el sistema?
- ¿Qué consecuencias puede tener almacenar datos inválidos?

Solución (modelo)

Aplicación de técnicas de validación y captura de datos en el Sistema ConstruSoft Ticket

1. Propósito de la actividad

Orientar el desarrollo de la Semana 06 mediante la implementación de técnicas de validación y captura de datos en el sistema ConstruSoft Ticket, manteniendo la arquitectura limpia construida en semanas anteriores.

En esta sesión, el estudiante debe comprender que:

- Validar datos no es solo una tarea visual;
- El frontend mejora la experiencia del usuario;
- El backend protege realmente el sistema;
- La calidad del software depende de la calidad de los datos ingresados.

2. Estado esperado antes de iniciar

Componente	Estado esperado
Backend	API funcional ASP.NET Core
Frontend	React + Vite operativo
Persistencia	PostgreSQL funcionando
Arquitectura	Clean Architecture aplicada
Flujo	React :: API :: PostgreSQL
Formulario	Registro de tickets operativo

3. Objetivo técnico de la semana

Implementar validaciones en:

- Frontend (React)
- Backend (.NET)

Para garantizar:

- Obligatoriedad;
- Longitud mínima;
- Longitud máxima;

- Mensajes claros;
- Control de errores;
- Consistencia de datos.

4. Archivos que se modificarán

Archivo	Acción
backend/ConstruSoftTicket.Application/DTOs/CreateTicketDto.cs	Agregar DataAnnotations.
backend/ConstruSoftTicket.API/Controllers/TicketController.cs	Validar ModelState.
frontend/src/pages/CreateTicket.jsx	Agregar validaciones avanzadas.
frontend/src/components/InputField.jsx	Mejorar soporte visual.

5. Archivos que se conservarán

Archivo	Acción
Ticket.cs	No modificar
TicketService.cs	No modificar
TicketRepository.cs	No modificar
AppDbContext.cs	No modificar
ticketService.js	No modificar

6. Desarrollo paso a paso

Paso 1. Levantar el backend

Ruta exacta

/Users/jorgealbertovegaflores/Documents/ConstruSoftTicket/backend/ConstruSoftTicket.API

Comandos

macOS	Windows 11
cd /Users/jorgealbertovegaflores/Documents/ConstruSoftTicket/backend/ConstruSoftTicket.API dotnet run	cd C:\Ruta\ConstruSoftTicket\backend\ConstruSoftTicket.API dotnet run

Resultado esperado

Debe mostrarse una URL similar a:

http://localhost:5153

También debe abrir Swagger en:
http://localhost:5153/swagger/index.html

Paso 2. Levantar el frontend
Abrir una segunda terminal.

Ruta exacta en macOS
/Users/jorgealbertovegaflores/Documents/ConstruSoftTicket/frontend

macOS
cd /Users/jorgealbertovegaflores/Documents/ConstruSoftTicket/frontend
npm run dev

Windows 11
cd C:\Ruta\ConstruSoftTicket\frontend
npm run dev

Resultado esperado
Debe mostrarse:
http://localhost:5173/

7. **Implementar validaciones en backend.**
   Paso 3. Modificar CreateTicketDto.cs
   Ruta exacta
   backend/ConstruSoftTicket.Application/DTOs/CreateTicketDto.cs

   Acción
   Reemplazar completamente el contenido actual por:

CreateTicketDto.cs

backend > ConstruSoftTicket.Application > DTOs > CreateTicketDto.cs

```
1   using System.ComponentModel.DataAnnotations;
2
3   namespace ConstruSoftTicket.Application.DTOs;
4
5   public class CreateTicketDto
6   {
7       [Required(ErrorMessage = "El título es obligatorio.")]
8       [MinLength(5, ErrorMessage = "El título debe tener al menos 5 caracteres.")]
9       [MaxLength(100, ErrorMessage = "El título no puede superar los 100 caracteres.")]
10      public string Titulo { get; set; } = string.Empty;
11
12      [Required(ErrorMessage = "La descripción es obligatoria.")]
13      [MinLength(10, ErrorMessage = "La descripción debe tener al menos 10 caracteres.")]
14      [MaxLength(500, ErrorMessage = "La descripción no puede superar los 500 caracteres.")]
15      public string Descripcion { get; set; } = string.Empty;
16  }
```

Explicación

Aquí se introducen las primeras técnicas formales de validación utilizando:

- Required
- MinLength
- MaxLength

Esto protege al sistema incluso si alguien evita las validaciones del frontend.

Paso 4. Modificar TicketController.cs

Ruta exacta

backend/ConstruSoftTicket.API/Controllers/TicketController.cs

Acción

Reemplazar completamente el contenido por:

Código

CreateTicketDto.cs | TicketController.cs

backend > ConstruSoftTicket.API > Controllers > TicketController.cs

```
1   using ConstruSoftTicket.Application.DTOs;
2   using ConstruSoftTicket.Application.Interfaces;
3   using Microsoft.AspNetCore.Mvc;
4
5   namespace ConstruSoftTicket.API.Controllers;
6
7   [ApiController]
8   [Route("api/[controller]")]
9   public class TicketController : ControllerBase
10  {
11      private readonly ITicketService ticketService;
12
13      public TicketController(ITicketService ticketService)
14      {
15          this.ticketService = ticketService;
16      }
17
18      [HttpPost]
19      public IActionResult Crear([FromBody] CreateTicketDto dto)
20      {
21          if (!ModelState.IsValid)
22          {
23              return BadRequest(ModelState);
24          }
25
26          ticketService.CrearTicket(dto);
27
28          return Ok(new
29          {
30              mensaje = "Ticket registrado correctamente."
31          });
32      }
33  }
```

Explicación docente

Con ModelState.IsValid, el backend:

- Valida automáticamente el DTO;

- Evita guardar información inválida;
- Devuelve errores HTTP adecuados.

Paso 5. Reiniciar backend

Después de modificar backend:

Ruta exacta

backend/ConstruSoftTicket.API

Acción

Detener ejecución actual:

Ctrl + C

Luego volver a ejecutar:

dotnet run

8. **Mejorar validaciones en frontend.**

   Paso 6. Mejorar InputField.jsx

   Ruta exacta

   frontend/src/components/InputField.jsx

   Acción

   Reemplazar completamente el contenido por:

InputField.jsx

frontend > src > components > InputField.jsx > InputField

```jsx
export default function InputField({
  label,
  name,
  value,
  onChange,
  placeholder,
  type = "text",
  required = false,
  error = ""
}) {
  return (
    <div>
      <label htmlFor={name}>{label}</label>

      <input
        id={name}
        name={name}
        type={type}
        value={value}
        placeholder={placeholder}
        required={required}
        onChange={onChange}
      />

      {error && (
        <p>
          {error}
        </p>
      )}
    </div>
  );
}
```

Explicación docente

Ahora el componente permite mostrar mensajes de error específicos por campo.

Paso 7. Mejorar CreateTicket.jsx

Ruta exacta

frontend/src/pages/CreateTicket.jsx

Acción

Reemplazar completamente el contenido actual por:

CreateTicket.jsx

frontend > src > pages > CreateTicket.jsx > CreateTicket

```jsx
import { useState } from "react";
import InputField from "../components/InputField";
import FormMessage from "../components/FormMessage";
import { createTicket } from "../services/ticketService";

export default function CreateTicket() {
  const [formData, setFormData] = useState({
    titulo: "",
    descripcion: ""
  });

  const [errors, setErrors] = useState({});

  const [message, setMessage] = useState("");
  const [messageType, setMessageType] = useState("");

  const [isSubmitting, setIsSubmitting] = useState(false);

  const handleChange = (event) => {
    const { name, value } = event.target;

    setFormData({
      ...formData,
      [name]: value
    });

    setErrors({
      ...errors,
      [name]: ""
    });
  };

```

```
33  const validateForm = () => {
34    const newErrors = {};
35
36    if (!formData.titulo.trim()) {
37      newErrors.titulo = "El título es obligatorio.";
38    } else if (formData.titulo.trim().length < 5) {
39      newErrors.titulo =
40        "El título debe tener al menos 5 caracteres.";
41    }
42
43    if (!formData.descripcion.trim()) {
44      newErrors.descripcion = "La descripción es obligatoria.";
45    } else if (formData.descripcion.trim().length < 10) {
46      newErrors.descripcion =
47        "La descripción debe tener al menos 10 caracteres.";
48    }
49
50    setErrors(newErrors);
51
52    return Object.keys(newErrors).length === 0;
53  };
54
55  const handleSubmit = async (event) => {
56    event.preventDefault();
57
58    setMessage("");
59    setMessageType("");
60
61    if (!validateForm()) {
62      return;
63    }
64
```

```
    try {
      setIsSubmitting(true);

      const result = await createTicket({
        titulo: formData.titulo.trim(),
        descripcion: formData.descripcion.trim()
      });

      setMessageType("success");
      setMessage(result.mensaje);

      setFormData({
        titulo: "",
        descripcion: ""
      });

      setErrors({});
    } catch (error) {
      console.error(error);

      setMessageType("error");
      setMessage(
        "No se pudo registrar el ticket. Verifique la información ingresada."
      );
    } finally {
      setIsSubmitting(false);
    }
  };

```

```
  return (
    <main>
      <h1>Registro de Ticket</h1>

      <form onSubmit={handleSubmit}>
        <InputField
          label="Título del ticket"
          name="titulo"
          value={formData.titulo}
          placeholder="Ejemplo: Equipo no enciende"
          required={true}
          onChange={handleChange}
          error={errors.titulo}
        />

        <div>
          <label htmlFor="descripcion">
            Descripción de la incidencia
          </label>

          <textarea
            id="descripcion"
            name="descripcion"
            value={formData.descripcion}
            placeholder="Describa detalladamente el problema"
            rows="5"
            onChange={handleChange}
          />

          {errors.descripcion && (
            <p>
              {errors.descripcion}
            </p>
          )}
        </div>


        <button type="submit" disabled={isSubmitting}>
          {isSubmitting ? "Registrando..." : "Registrar ticket"}
        </button>
      </form>

      <FormMessage type={messageType} message={message} />
    </main>
  );
}
```

9. **Pruebas funcionales**

Paso 8. Validar título corto

Acción

Ingresar:

abc

Como título.

Resultado esperado

Debe mostrarse:

El título debe tener al menos 5 caracteres.

Paso 9. Validar descripción vacía.

Acción

Dejar descripción vacía.

Resultado esperado

Debe mostrarse:

La descripción es obligatoria.

Paso 10. Validar registro correcto.

Datos sugeridos

Título

Equipo no inicia.

Descripción

El equipo del laboratorio 02 no responde al presionar el botón de encendido.

Resultado esperado

- Mensaje de éxito;
- Limpieza del formulario;
- Registro almacenado en PostgreSQL.

10. **Verificar en PostgreSQL.**

Paso 11. Entrar a la base de datos

macOS

psql -d ConstruSoftTicketDB

Si se requiere usuario:

psql -U TU_USUARIO -d ConstruSoftTicketDB

Windows 11

psql -U TU_USUARIO -d ConstruSoftTicketDB

Paso 12. Consultar registros.

```
SELECT "Titulo", "Descripcion", "Estado"
FROM "Tickets";
```

11. **Explicación técnica**

¿Por qué validar también en backend?

Porque el frontend puede ser evitado.

Por ejemplo:

- Usando Postman;
- Modificando JavaScript;
- Consumiendo directamente el endpoint.

Por eso el backend es la última barrera de protección del sistema.

Diferencia entre frontend y backend

Frontend	Backend
Mejora la experiencia del usuario.	Protege realmente el sistema
Validación visual.	Validación definitiva
Mensajes inmediatos.	Seguridad e integridad

12. Errores frecuentes y solución

Error	Causa	Solución
Failed to fetch	API apagada	Verificar dotnet run
CORS error	Mala configuración	Revisar Program.cs
No aparecen errores.	error={errors.titulo} faltante	Revisar InputField
Backend acepta datos inválidos	DTO sin DataAnnotations	Revisar CreateTicketDto.cs
Pantalla en blanco	Error de importación	Revisar consola navegador.

13. Resultado esperado de la semana

El sistema debe:

- Validar datos en frontend.
- Validar datos en backend.
- Mostrar mensajes claros.
- Impedir registros inválidos.
- Almacenar solo información válida.
- Mantener arquitectura limpia.

14. Resultado esperado de la semana

Criterio	Indicador
Validación frontend	Detecta errores antes del envío.
Validación backend	Protege la API.
Calidad de datos	Solo registra información válida.
UX	Mensajes claros y útiles
Arquitectura	Mantiene separación de responsabilidades.
Funcionamiento	Flujo React → API → PostgreSQL operativo

"La validación de datos no debe considerarse un complemento del sistema, sino una responsabilidad esencial de la ingeniería de software, porque protege tanto al usuario como a la integridad de la solución."

Jorge A. Vega Flores

UPLA

Construcción de software

Semana 07
(Laboratorio)

Reutilización, robustez y mejora visual de los módulos de entrada de datos en el Sistema ConstruSoft Ticket

ASIGNATURA
Construcción de software

ACTIVIDAD 01

“La validación de datos no debe considerarse un complemento del sistema, sino una responsabilidad esencial de la ingeniería de software, porque protege tanto al usuario como a la integridad de la solución.”
— Jorge A. Vega Flores

1. **Objetivo**

   Implementar mejoras estructurales, visuales y funcionales en el módulo de registro de tickets del sistema ConstruSoft Ticket, incorporando reutilización de validaciones, manejo estructurado de errores, respuestas HTTP más robustas y una mejora progresiva de la experiencia visual del usuario.

2. **Contexto**

   En las semanas anteriores, el sistema logró:

   - Implementar el flujo React :: API :: postgresql;
   - Validar datos en frontend y backend;
   - Proteger el sistema frente a entradas inválidas;
   - Estructurar correctamente las capas del proyecto.

   Sin embargo, el módulo de entrada aún presenta aspectos que deben fortalecerse:

   - Las respuestas HTTP no están completamente estructuradas;
   - Las validaciones podrían reutilizarse mejor;
   - Los mensajes de error aún son básicos;
   - La interfaz visual todavía es funcional, pero poco profesional;

- El sistema no normaliza adecuadamente algunos datos ingresados.

En sistemas reales, los módulos de entrada deben ser:

- Robustos;
- Reutilizables;
- Claros para el usuario;
- Seguros;
- Visualmente organizados.

Tal como señalan Robert C. Martin y Martin Fowler, la mantenibilidad y calidad del software dependen tanto de la arquitectura como de la forma en que el sistema gestiona sus datos y su interacción con el usuario.

3. **Problema de la semana**

   ¿Cómo mejorar el módulo de entrada de datos del sistema ConstruSoft Ticket para hacerlo más robusto, reutilizable, seguro y visualmente claro, manteniendo la arquitectura limpia implementada en semanas anteriores?

4. **Condiciones y restricciones**

   El estudiante debe considerar:

   - Mantener la arquitectura limpia;
   - No romper el flujo react :: api :: postgresql;
   - Reutilizar componentes cuando sea posible;
   - Mantener separación de responsabilidades;
   - Aplicar mejoras visuales progresivas;

- Mantener coherencia entre frontend y backend;
- Utilizar validaciones consistentes;
- Evitar duplicación innecesaria de lógica.

5. Actividades / Preguntas orientadoras

1. Reutilización de validaciones
   - ¿Qué validaciones podrían reutilizarse en otros formularios?
   - ¿Qué ventajas tiene centralizar reglas de validación?

2. Robustez del backend
   - ¿Cómo debería responder una API ante errores?
   - ¿Qué diferencia existe entre una respuesta HTTP simple y una estructurada?

3. Manejo de errores
   - ¿Cómo debe reaccionar el frontend ante errores del backend?
   - ¿Qué información debe mostrarse al usuario y cuál no?

4. Seguridad de entrada de datos
   - ¿Qué riesgos existen si no se normalizan los datos?
   - ¿Por qué es importante limpiar espacios innecesarios?
   - ¿Qué problemas puede generar almacenar información inconsistente?

5. Mejora visual del módulo
   - ¿Cómo influye el diseño visual en la captura de datos?
   - ¿Qué elementos mejoran la experiencia del usuario?
   - ¿Cómo ayuda una interfaz organizada a reducir errores?

6. Arquitectura y mantenibilidad
    - ¿Cómo ayuda la reutilización de componentes a mantener el sistema?
    - ¿Qué problemas aparecen cuando el frontend crece sin organización?

6. Entregable

El estudiante deberá presentar:

1. Sistema actualizado con:
    - Manejo estructurado de errores;
    - Validaciones reutilizables;
    - Mejoras visuales;
    - Respuestas http más robustas.
2. Capturas de:
    - Mensajes de error;
    - Validaciones visuales;
    - Interfaz mejorada;
    - Respuestas exitosas;
    - Errores controlados.
3. Documento breve que explique:
    - Mejoras implementadas;
    - Principios aplicados;
    - Beneficios obtenidos;
    - Reflexión sobre calidad y experiencia de usuario.

7. Reflexión final

El estudiante deberá responder:

- ¿Por qué la experiencia visual también forma parte de la calidad del software?
- ¿Qué ventajas aporta reutilizar componentes y validaciones?
- ¿Cómo influye el manejo correcto de errores en la confiabilidad del sistema?
- ¿Qué diferencia existe entre “hacer funcionar” un sistema y construirlo correctamente?

Solución (modelo)

Reutilización, robustez y mejora visual de los módulos de entrada de datos en el Sistema ConstruSoft Ticket

1. Propósito de la actividad

Orientar el desarrollo de la Semana 07 mediante la mejora estructural, visual y funcional del módulo de registro de tickets del sistema ConstruSoft Ticket, fortaleciendo:

- Reutilización de componentes;
- Manejo estructurado de errores;
- Validaciones reutilizables;
- Respuestas http consistentes;
- Normalización de datos;
- Experiencia visual del usuario;
- Organización del frontend.

Esta semana marca el inicio de una transición importante:

El sistema deja de ser solamente "funcional" y empieza a evolucionar hacia una solución más mantenible, robusta y profesional.

2. Estado esperado antes de iniciar

El proyecto debe contar con:

Componente	Estado esperado
Backend	ASP.NET Core funcionando
Frontend	React + Vite operativo
Base de datos	PostgreSQL operativo
Validaciones	Frontend + backend
Arquitectura	Clean Architecture aplicada
Flujo	React > API > PostgreSQL
Formulario	Registro de tickets validado

3. Objetivo técnico de la semana

Implementar:

- Respuestas HTTP estructuradas;
- Manejo controlado de errores;
- Normalización básica de datos;
- Componentes visuales reutilizables;
- Primeros estilos funcionales con Tailwind CSS;
- Mejora visual del formulario;
- Organización más limpia del frontend.

4. Alcance técnico de la semana

Backend

Se trabajará:

- Respuestas HTTP consistentes;
- Estructura estándar de respuestas;
- Control básico de excepciones;
- Normalización de cadenas.

Frontend

Se trabajará:

- Tailwind CSS;
- Reutilización visual;
- Mejora UX;
- Feedback visual;
- Mensajes de error estilizados;
- Estados visuales del formulario.

5. Archivos que se modificarán

Archivo	Acción
backend/ConstruSoftTicket.Application/DTOs/CreateTicketDto.cs	Normalización básica
backend/ConstruSoftTicket.API/Controllers/TicketController.cs	Respuestas estructuradas
frontend/src/pages/CreateTicket.jsx	Reestructuración visual
frontend/src/components/InputField.jsx	Mejora visual
frontend/src/components/FormMessage.jsx	Mejora visual
frontend/src/index.css	Tailwind CSS

frontend/package.json	Dependencias Tailwind

6. Archivos que se crearán

Archivo	Propósito
frontend/tailwind.config.js	Configuración Tailwind
frontend/postcss.config.js	Configuración PostCSS
frontend/src/components/Button.jsx	Botón reutilizable
frontend/src/utils/normalizeText.js	Normalización reutilizable

7. Desarrollo paso a paso

Paso 1. Abrir terminal del frontend

macOS

cd /Users/jorgealbertovegaflores/Documents/ConstruSoftTicket/frontend

Windows 11

cd C:\Ruta\ConstruSoftTicket\frontend

Paso 2. Instalar Tailwind CSS

Comando

npm install -D tailwindcss@3.4.3 postcss autoprefixer

Paso 3. Inicializar Tailwind

Comando

npx tailwindcss init -p

Resultado esperado

Se crean:

tailwind.config.js

postcss.config.js

Paso 4. Configurar tailwind.config.js

Ruta exacta

frontend/tailwind.config.js

Acción

Reemplazar completamente el contenido por:

Código

Paso 5. Configurar index.css

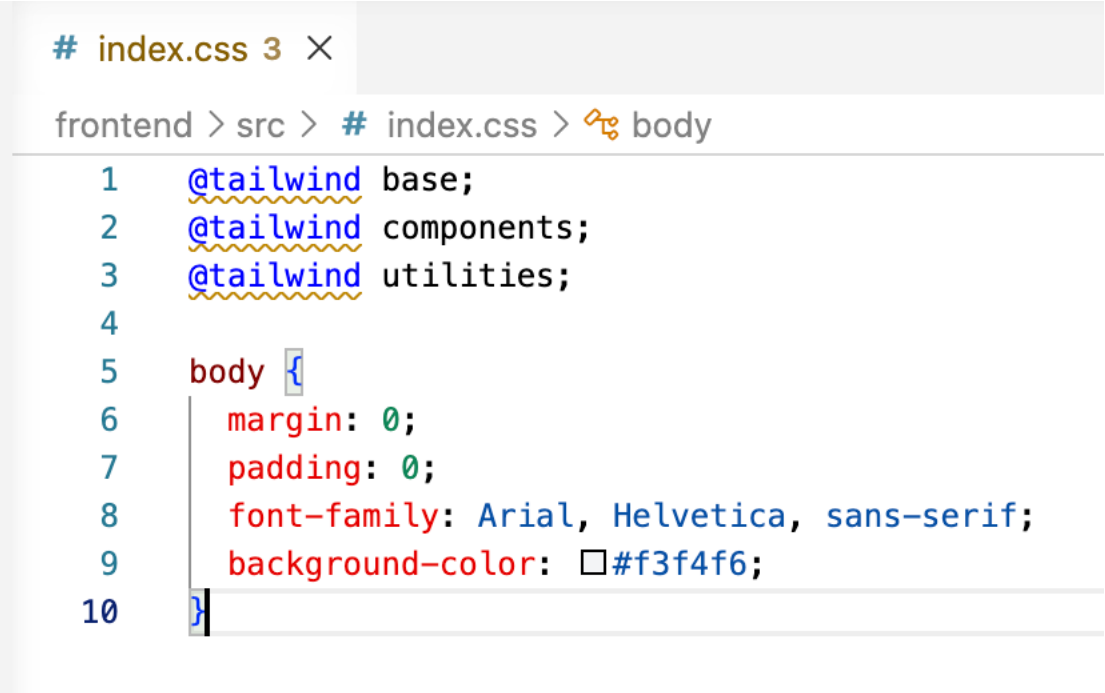
Ruta exacta

frontend/src/index.css

Acción

Reemplazar completamente el contenido por:

Código:



Paso 6. Reiniciar frontend

Detener el frontend

Ctrl + C

Volver a ejecutar

npm run dev

Explicación docente

Ahora el componente permite mostrar mensajes de error específicos por campo.

Normalización de datos

Paso 7. Crear utilidad de normalización

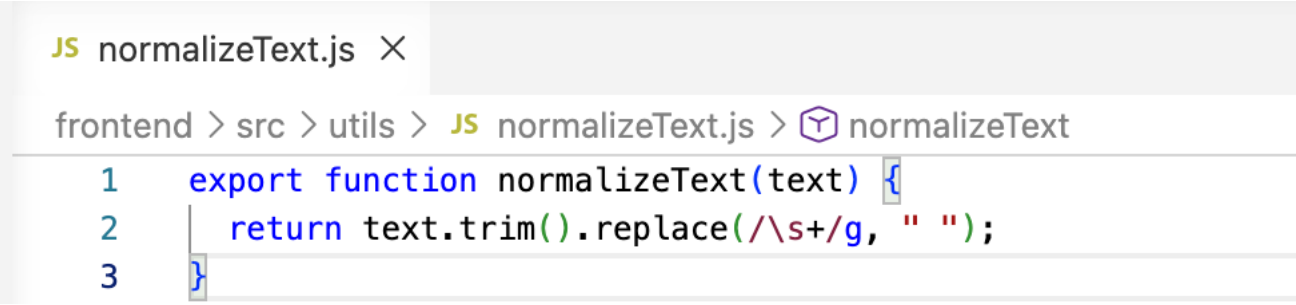
Ruta exacta

frontend/src/utils/normalizeText.js

Acción

Si la carpeta utils no existe, crearla.

Código:



Explicación

Esta función:

- Elimina espacios innecesarios;
- Evita registros inconsistentes;
- Mejora calidad de datos.

Paso 8. Mejorar TicketController.cs

Ruta exacta:

backend/ConstruSoftTicket.API/Controllers/TicketController.cs

Acción

Reemplazar completamente el contenido por:

TicketController.cs

backend > ConstruSoftTicket.API > Controllers > TicketController.cs > { } ConstruSoftTic

```
5   namespace ConstruSoftTicket.API.Controllers;
9   public class TicketController : ControllerBase
19      public IActionResult Crear([FromBody] CreateTicketDto dto)
24          {
25              return BadRequest(new
26              {
27                  success = false,
28                  message = "Datos inválidos.",
29                  errors = ModelState
30              });
31          }
32
33          dto.Titulo = dto.Titulo.Trim();
34          dto.Descripcion = dto.Descripcion.Trim();
35
36          ticketService.CrearTicket(dto);
37
38          return Ok(new
39          {
40              success = true,
41              message = "Ticket registrado correctamente."
42          });
43      }
44      catch (Exception ex)
45      {
46          return StatusCode(500, new
47          {
48              success = false,
49              message = "Ocurrió un error interno.",
50              detail = ex.Message
51          });
52      }
53  }
54  }
```

Explicación

Ahora la API:

- Devuelve respuestas estructuradas;
- Diferencia éxito/error;

- Controla excepciones;
- Normaliza datos.

Paso 9. Reiniciar backend

Ruta exacta

backend/ConstruSoftTicket.API

Comando

dotnet run

Paso 10. Crear componente Button

Ruta exacata

frontend/src/components/Button.jsx

Código:

Button.jsx

frontend > src > components > Button.jsx > Button

```jsx
export default function Button({
  children,
  disabled = false,
  type = "button"
}) {
  return (
    <button
      type={type}
      disabled={disabled}
      className="
        bg-blue-600
        hover:bg-blue-700
        text-white
        font-semibold
        px-4
        py-2
        rounded
        transition
        duration-200
        disabled:bg-gray-400
      "
    >
      {children}
    </button>
  );
}
```

Paso 11. Mejorar InputField.jsx

Ruta exacta

frontend/src/components/InputField.jsx

Reemplazar completamente por:

InputField.jsx

frontend > src > components > InputField.jsx > InputField

```jsx
export default function InputField({
  label,
  name,
  value,
  onChange,
  placeholder,
  type = "text",
  required = false,
  error = ""
}) {
  return (
    <div className="mb-4">
      <label
        htmlFor={name}
        className="block mb-2 font-medium text-gray-700"
      >
        {label}
      </label>
```

```
19
20        <input
21          id={name}
22          name={name}
23          type={type}
24          value={value}
25          placeholder={placeholder}
26          required={required}
27          onChange={onChange}
28          className={`
29            w-full
30            border
31            rounded
32            px-3
33            py-2
34            outline-none
35            focus:ring-2
36            ${
37              error
38                ? "border-red-500 focus:ring-red-300"
39                : "border-gray-300 focus:ring-blue-300"
40            }
41          `}
42        />
43
44        {error && (
45          <p className="text-red-500 text-sm mt-1">
46            {error}
47          </p>
48        )}
49      </div>
50    );
51  }
```

Paso 12. Mejorar FormMessage.jsx

Ruta exacta

frontend/src/components/FormMessage.jsx

Reemplazar completamente con:

FormMessage.jsx

frontend > src > components > FormMessage.jsx > FormMessage

```jsx
export default function FormMessage({ type, message }) {
  if (!message) {
    return null;
  }

  return (
    <div
      className={`
        mt-4
        p-3
        rounded
        font-medium
        ${
          type === "success"
            ? "bg-green-100 text-green-700"
            : "bg-red-100 text-red-700"
        }
      `}
    >
      {message}
    </div>
  );
}
```

Paso 13. Mejorar CreateTicket.jsx

Ruta exacta:

frontend/src/pages/CreateTicket.jsx

Reemplazar completamente el contenido por:

CreateTicket.jsx

frontend > src > pages > CreateTicket.jsx > CreateTicket

```jsx
import { useState } from "react";

import InputField from "../components/InputField";
import FormMessage from "../components/FormMessage";
import Button from "../components/Button";

import { createTicket } from "../services/ticketService";
import { normalizeText } from "../utils/normalizeText";

export default function CreateTicket() {
  const [formData, setFormData] = useState({
    titulo: "",
    descripcion: ""
  });

  const [errors, setErrors] = useState({});

  const [message, setMessage] = useState("");
  const [messageType, setMessageType] = useState("");

  const [isSubmitting, setIsSubmitting] = useState(false);

  const handleChange = (event) => {
    const { name, value } = event.target;

    setFormData({
      ...formData,
      [name]: value
    });

    setErrors({
      ...errors,
      [name]: ""
    });
  };
```

```
36
37  const validateForm = () => {
38    const newErrors = {};
39
40    const titulo = normalizeText(formData.titulo);
41    const descripcion = normalizeText(formData.descripcion);
42
43    if (!titulo) {
44      newErrors.titulo = "El título es obligatorio.";
45    } else if (titulo.length < 5) {
46      newErrors.titulo =
47        "El título debe tener al menos 5 caracteres.";
48    }
49
50    if (!descripcion) {
51      newErrors.descripcion =
52        "La descripción es obligatoria.";
53    } else if (descripcion.length < 10) {
54      newErrors.descripcion =
55        "La descripción debe tener al menos 10 caracteres.";
56    }
57
58    setErrors(newErrors);
59
60    return Object.keys(newErrors).length === 0;
61  };
62
```

```
63  const handleSubmit = async (event) => {
64    event.preventDefault();
65
66    setMessage("");
67    setMessageType("");
68
69    if (!validateForm()) {
70      return;
71    }
72
73    try {
74      setIsSubmitting(true);
75
76      const result = await createTicket({
77        titulo: normalizeText(formData.titulo),
78        descripcion: normalizeText(formData.descripcion)
79      });
80
81      setMessageType("success");
82      setMessage(result.message);
83
84      setFormData({
85        titulo: "",
86        descripcion: ""
87      });
88
89      setErrors({});
90    } catch (error) {
91      console.error(error);
92
93      setMessageType("error");
94      setMessage(
95        "No se pudo registrar el ticket."
96      );
97    } finally {
98      setIsSubmitting(false);
99    }
100 };
```

```
101
102   return (
103     <main
104       className="
105         min-h-screen
106         flex
107         items-center
108         justify-center
109         bg-gray-100
110         px-4
111       "
112     >
113       <section
114         className="
115           bg-white
116           shadow-lg
117           rounded-lg
118           p-8
119           w-full
120           max-w-xl
121         "
122       >
123         <h1
124           className="
125             text-3xl
126             font-bold
127             mb-2
128             text-gray-800
129           "
130         >
131           Registro de Ticket
132         </h1>
133
```

```
134                 <p className="text-gray-600 mb-6">
135                   Complete correctamente la información de la incidencia téc
136                 </p>
137
138                 <form onSubmit={handleSubmit}>
139                   <InputField
140                     label="Título del ticket"
141                     name="titulo"
142                     value={formData.titulo}
143                     placeholder="Ejemplo: Equipo no inicia"
144                     required={true}
145                     onChange={handleChange}
146                     error={errors.titulo}
147                   />
148
149                   <div className="mb-4">
150                     <label
151                       htmlFor="descripcion"
152                       className="
153                         block
154                         mb-2
155                         font-medium
156                         text-gray-700
157                       "
158                     >
159                       Descripción de la incidencia
160                     </label>
161
```

```
<textarea
  id="descripcion"
  name="descripcion"
  value={formData.descripcion}
  placeholder="Describa detalladamente el problema"
  rows="5"
  onChange={handleChange}
  className={`
    w-full
    border
    rounded
    px-3
    py-2
    outline-none
    focus:ring-2
    ${
      errors.descripcion
        ? "border-red-500 focus:ring-red-300"
        : "border-gray-300 focus:ring-blue-300"
    }
  `}
/>
```

```
185                {errors.descripcion && (
186                  <p className="text-red-500 text-sm mt-1">
187                    {errors.descripcion}
188                  </p>
189                )}
190              </div>
191
192              <Button
193                type="submit"
194                disabled={isSubmitting}
195              >
196                {isSubmitting
197                  ? "Registrando..."
198                  : "Registrar ticket"}
199              </Button>
200            </form>
201
202            <FormMessage
203              type={messageType}
204              message={message}
205            />
206          </section>
207        </main>
208      );
209    }
```

8. **Validaciones funcionales**

   Paso 14. Validar formulario vacío

   Resultado esperado

   - Errores visuales en rojo;
   - Mensajes claros;
   - Botón funcional.

   Paso 15. Validar espacios innecesarios

   Entrada

   " Equipo no inicia "

   Resultado esperado

Se almacena como:

"Equipo no inicia"

Paso 16. Verificar respuesta HTTP estructurada

Abrir Swagger

http://localhost:5153/swagger/index.html

Probar POST /api/ticket

Respuesta esperada

```
{
  "success": true,
  "message": "Ticket registrado correctamente."
}
```

Paso 17. Verificar PostgreSQL

Comando

```
SELECT "Titulo", "Descripcion"
FROM "Tickets";
```

9. Explicación

¿Por qué introducir Tailwind ahora?

Porque el sistema ya:

- Funciona;
- Valida;
- Persiste datos.

Ahora puede evolucionar hacia:

- Mejor experiencia visual;
- Mantenibilidad visual;
- Diseño consistente.

¿Qué principios se fortalecen?

Principio	Aplicación

Reutilización	Componentes reutilizables
Separación de responsabilidades	Servicios, utilidades y componentes separados
Mantenibilidad	Código frontend organizado
UX	Feedback visual claro
Robustez	Respuestas HTTP estructuradas

10. Errores frecuentes

Error	Causa	Solución
Tailwind no funciona	Frontend no reiniciado	Ejecutar npm run dev nuevamente
Estilos no aplican	index.css incorrecto	Revisar imports Tailwind
normalizeText no encontrado	Ruta incorrecta	Verificar carpeta utils
result.message undefined	API no devuelve estructura correcta	Revisar TicketController.cs
Pantalla en blanco	Error JSX	Revisar consola navegador

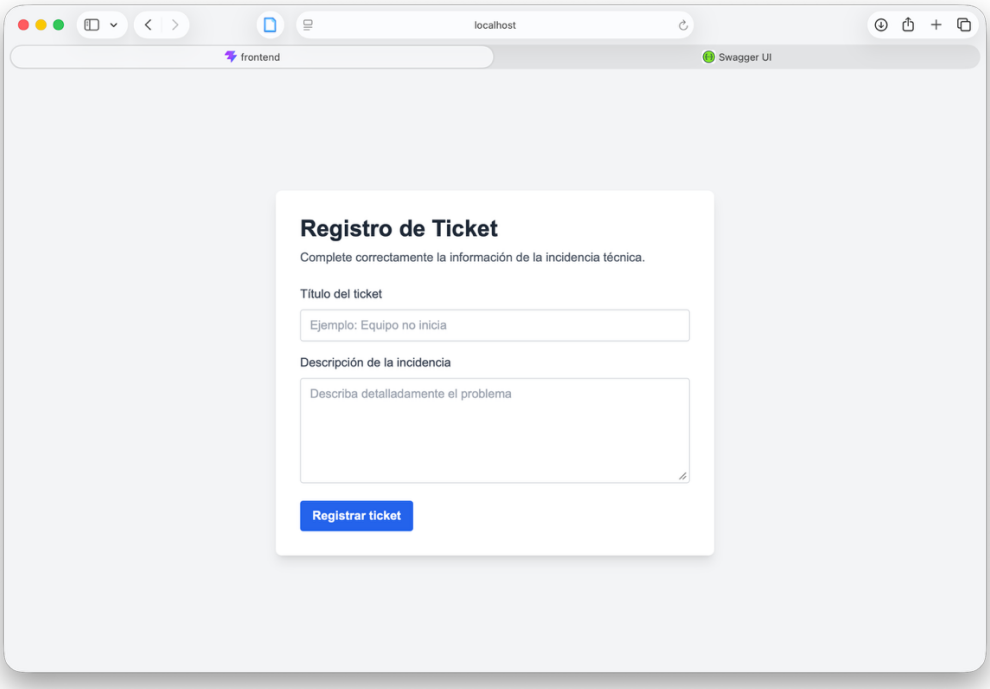
11. Resultado esperado de la semana

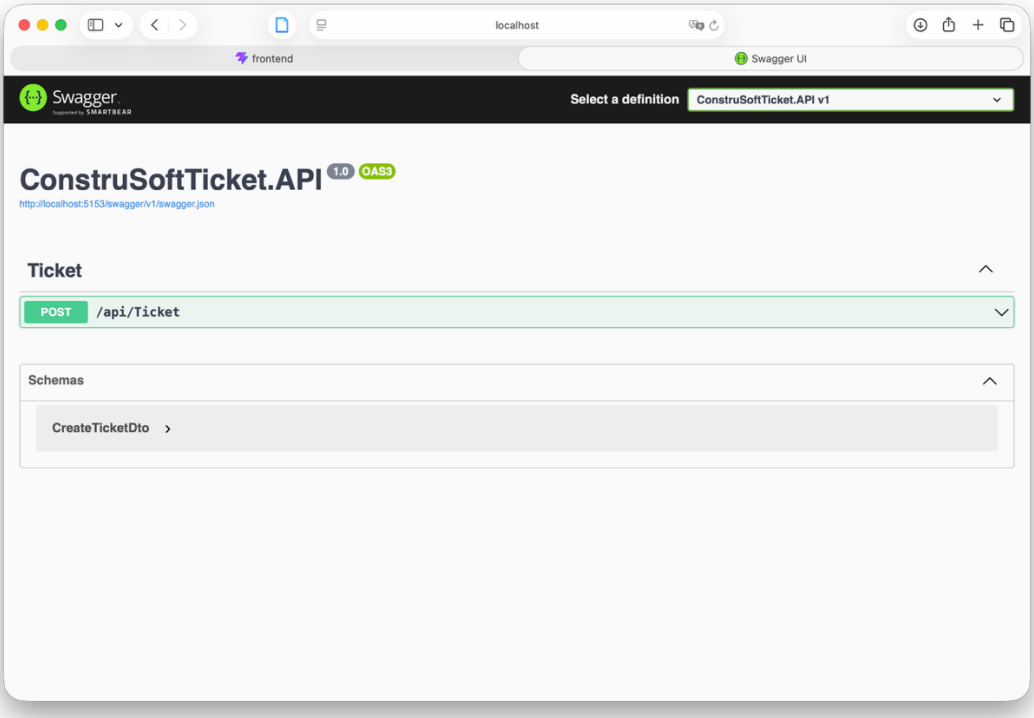
El sistema debe:

- Tener formulario estilizado
- Mostrar errores visuales claros
- Reutilizar componentes
- Normalizar datos
- Responder con estructura http consistente
- Mantener arquitectura limpia
- Seguir registrando tickets correctamente

12. Resultado esperado de la semana

Criterio	Indicador
UX/UI	Formulario visualmente claro
Reutilización	Uso de componentes reutilizables
Robustez	Manejo estructurado de errores
Normalización	Datos consistentes
Arquitectura	Organización frontend/backend
Funcionamiento	Flujo React → API → PostgreSQL operativo





“Construir software no consiste solo en hacer que funcione. También implica hacerlo mantenible, claro, reutilizable y comprensible para las personas que lo utilizan.”

Jorge A. Vega Flores

UPLA

Construcción de software

Semana 08
(Laboratorio)

Aplicar el refinamiento de código en módulos de entrada de datos en un caso práctico

ASIGNATURA
Construcción de software

ACTIVIDAD 01

“El refinamiento del software no consiste en agregar más código, sino en organizarlo de manera que sea comprensible, reutilizable y sostenible en el tiempo.”

— Jorge A. Vega Flores

1. **Objetivo**

   Aplicar técnicas de refinamiento de código al módulo de entrada de datos del sistema ConstruSoft Ticket, reorganizando componentes, servicios y estructura visual para evolucionar hacia una arquitectura SPA más mantenible, reutilizable y profesional.

2. **Contexto**

   En las semanas anteriores, el sistema logró:

   - Implementar el flujo React > API > postgresql;
   - Validar información en frontend y backend;
   - Estructurar respuestas HTTP;
   - Reutilizar componentes;
   - Incorporar estilos iniciales con Tailwind CSS.

   Sin embargo, conforme el sistema empieza a crecer, aparecen nuevos desafíos:

   - El frontend comienza a concentrar demasiadas responsabilidades;
   - La navegación todavía depende de una sola pantalla;
   - El código requiere mayor modularización;
   - La estructura visual aún no representa una spa empresarial;

- Algunos componentes todavía podrían reutilizarse mejor.

En proyectos reales, los sistemas evolucionan constantemente. Por ello, el refinamiento de código es una actividad esencial para mantener:

- Claridad;
- Mantenibilidad;
- Escalabilidad;
- Reutilización;
- Orden arquitectónico.

Tal como señalan Martin Fowler y Robert C. Martin, refactorizar y refinar código permite mejorar la estructura interna del software sin alterar su comportamiento funcional.

3. Problema de la semana

¿Cómo refinar el módulo de entrada de datos del sistema ConstruSoft Ticket para convertirlo en una estructura SPA más organizada, reutilizable y mantenible, sin afectar su funcionamiento actual?

4. Condiciones y restricciones

El estudiante debe considerar:

- Mantener el flujo funcional React > API > postgresql;
- No romper la arquitectura limpia implementada;
- Mantener las validaciones actuales;
- Reorganizar componentes y estructura del frontend;
- Mantener coherencia visual;

- Separar responsabilidades adecuadamente;
- Preparar el sistema para futuras páginas.

5. Actividades / Preguntas orientadoras

1. Análisis del código actual
   - ¿Qué responsabilidades están concentradas en un mismo archivo?
   - ¿Qué partes del frontend podrían reutilizarse mejor?
   - ¿Qué problemas aparecen cuando un sistema crece sin organización?

2. Refinamiento de componentes
   - ¿Qué ventajas tiene separar layouts, páginas y componentes?
   - ¿Cómo ayuda la reutilización al mantenimiento del sistema?

3. Evolución hacia SPA
   - ¿Qué caracteriza a una SPA moderna?
   - ¿Por qué es importante separar navegación y contenido?
   - ¿Cómo influye la organización visual en la experiencia del usuario?

4. Refinamiento visual
   - ¿Qué elementos visuales mejoran la claridad del sistema?
   - ¿Cómo ayuda una estructura empresarial a la usabilidad?

5. Arquitectura y mantenibilidad
   - ¿Cómo ayuda el refinamiento de código a la escalabilidad?

- ¿Por qué una arquitectura desordenada dificulta futuras funcionalidades?

6. Entregable

El estudiante deberá presentar:

1. Sistema refinado con:
   - Estructura SPA inicial;
   - Componentes organizados;
   - Layout principal;
   - Navegación básica;
   - Frontend reorganizado.
2. Capturas de:
   - Interfaz refinada;
   - Estructura spa;
   - Formulario integrado en layout;
   - Navegación visual.
3. Documento breve explicando:
   - Refinamientos aplicados;
   - Mejoras estructurales;
   - Beneficios obtenidos;
   - Reflexión sobre mantenibilidad.

7. Reflexión final

El estudiante deberá responder:

- ¿Qué diferencia existe entre un sistema funcional y uno mantenible?
- ¿Por qué el refinamiento de código es parte de la ingeniería de software?

- ¿Cómo ayuda una SPA bien estructurada al crecimiento del sistema?
- ¿Qué riesgos aparecen cuando un proyecto crece sin organización?

Semana 08

Solución (modelo)

Refinamiento de código y estructuración SPA del módulo de entrada de datos en el Sistema ConstruSoft Ticket

1. Propósito de la actividad

Orientar el desarrollo de la Semana 08 mediante la aplicación de técnicas de refinamiento de código al módulo de entrada de datos del sistema ConstruSoft Ticket, reorganizando la estructura del frontend para evolucionar progresivamente hacia una SPA empresarial mantenible y reutilizable.

En esta semana, el objetivo ya no es únicamente “hacer funcionar” el sistema, sino mejorar:

- Organización;
- Legibilidad;
- Reutilización;
- Separación de responsabilidades;
- Estructura visual;
- Mantenibilidad.

2. Estado esperado antes de iniciar

El proyecto debe contar con:

Componente	Estado esperado
Backend	ASP.NET Core operativo
Frontend	React + Vite funcionando
Persistencia	PostgreSQL operativo
Validaciones	Frontend + backend
Tailwind CSS	Configurado
Arquitectura	Clean Architecture aplicada
Formulario	Estilizado y validado

3. Objetivo técnico de la semana

Refinar el frontend del sistema mediante:

- Reorganización de carpetas;
- Separación de layouts;
- Creación de estructura spa;
- Reutilización visual;
- Mejora del diseño empresarial;
- Refinamiento de componentes;
- Preparación para futuras páginas.

4. Resultado esperado

Al finalizar la semana, el sistema deberá tener:

- Estructura SPA básica.
- Layout principal reutilizable
- Sidebar lateral
- Cabecera principal
- Formulario integrado en layout empresarial
- Código frontend más organizado
- Navegación preparada para crecimiento futuro

5. Arquitectura frontend esperada

La estructura final deberá quedar así:

6. Archivos que se crearán

Archivo	Propósito
Sidebar.jsx	Menú lateral SPA
Header.jsx	Encabezado principal
PageContainer.jsx	Contenedor reutilizable
MainLayout.jsx	Layout general SPA

7. Archivos que se modificarán

Archivo	Acción
CreateTicket.jsx	Integrarlo al layout SPA
App.jsx	Simplificar renderizado
index.css	Ajustes globales menores

8. **Desarrollo paso a paso**

Paso 1. Abrir terminal del frontend

macOS

cd /Users/jorgealbertovegaflores/Documents/ConstruSoftTicket/frontend

Windows 11

cd C:\Ruta\ConstruSoftTicket\frontend

Ejecutar frontend

npm run dev

Paso 2. Crear carpeta layouts

Ruta exacta

frontend/src/

Acción

Crear carpeta:

layouts

Paso 3. Crear Sidebar.jsx

Ruta exacta:

frontend/src/components/Sidebar.jsx

Código:

Sidebar.jsx

frontend > src > components > Sidebar.jsx > Sidebar

```jsx
export default function Sidebar() {
  return (
    <aside
      className="
        w-64
        bg-gray-900
        text-white
        min-h-screen
        p-6
      "
    >
      <h2 className="text-2xl font-bold mb-8">
        ConstruSoft
      </h2>

      <nav>
        <ul className="space-y-4">
          <li>
            <button
              className="
                w-full
                text-left
                hover:text-blue-300
                transition
              "
            >
              Registro de Tickets
            </button>
          </li>

          <li>
            <button
              className="
                w-full
                text-left
                text-gray-400
              "
            >
              Próximamente...
            </button>
          </li>
        </ul>
```

```
43          </nav>
44        </aside>
45      );
46    }
```

Explicación

El sidebar representa:

- Navegación empresarial;
- Separación visual;
- Inicio de spa modular.

Paso 4. Crear Header.jsx

Ruta exacta

frontend/src/components/Header.jsx

Código

Header.jsx

frontend > src > components > Header.jsx > Header

```jsx
export default function Header() {
  return (
    <header
      className="
        bg-white
        shadow
        px-6
        py-4
        flex
        justify-between
        items-center
      "
    >
      <div>
        <h1 className="text-2xl font-bold text-gray-800">
          Sistema ConstruSoft Ticket
        </h1>

        <p className="text-sm text-gray-500">
          Gestión de incidencias técnicas
        </p>
      </div>

      <div
        className="
          bg-blue-100
          text-blue-700
          px-3
          py-1
          rounded-full
          text-sm
          font-medium
        "
      >
        SPA v1.0
      </div>
    </header>
  );
}
```

Paso 5. Crear PageContainer.jsx

Ruta exacta

frontend/src/components/PageContainer.jsx

Código:

PageContainer.jsx

frontend > src > components > PageContainer.jsx > PageContainer

```jsx
export default function PageContainer({
  title,
  children
}) {
  return (
    <section
      className="
        bg-white
        rounded-lg
        shadow
        p-8
      "
    >
      <h2
        className="
          text-2xl
          font-bold
          text-gray-800
          mb-6
        "
      >
        {title}
      </h2>

      {children}
    </section>
  );
}
```

Explicación

Este componente evita repetir:

- Sombras;
- Padding;
- Estilos de contenedor;
- Estructura visual.

Paso 6. Crear MainLayout.jsx

Ruta exacta

frontend/src/layouts/MainLayout.jsx

Código

MainLayout.jsx

frontend > src > layouts > MainLayout.jsx > MainLayout

```jsx
import Sidebar from "../components/Sidebar";
import Header from "../components/Header";

export default function MainLayout({
  children
}) {
  return (
    <div className="flex min-h-screen bg-gray-100">
      <Sidebar />

      <div className="flex-1 flex flex-col">
        <Header />

        <main className="p-6">
          {children}
        </main>
      </div>
    </div>
  );
}
```

Explicación

Aquí aparece por primera vez:

- Estructura SPA
- Separación navegación/contenido
- Layout reutilizable

Paso 7. Modificar CreateTicket.jsx

Ruta exacta

frontend/src/pages/CreateTicket.jsx

Acción

Reemplazar completamente el contenido por:

Código:

CreateTicket.jsx

frontend > src > pages > CreateTicket.jsx > CreateTicket

```jsx
import { useState } from "react";

import InputField from "../components/InputField";
import FormMessage from "../components/FormMessage";
import Button from "../components/Button";
import PageContainer from "../components/PageContainer";

import MainLayout from "../layouts/MainLayout";

import { createTicket } from "../services/ticketService";
import { normalizeText } from "../utils/normalizeText";

export default function CreateTicket() {
  const [formData, setFormData] = useState({
    titulo: "",
    descripcion: ""
  });

  const [errors, setErrors] = useState({});

  const [message, setMessage] = useState("");
  const [messageType, setMessageType] = useState("");

  const [isSubmitting, setIsSubmitting] = useState(false);

  const handleChange = (event) => {
    const { name, value } = event.target;

    setFormData({
      ...formData,
      [name]: value
    });

    setErrors({
      ...errors,
      [name]: ""
    });
  };
```

```
39
40  const validateForm = () => {
41    const newErrors = {};
42
43    const titulo = normalizeText(formData.titulo);
44    const descripcion = normalizeText(formData.descripcion);
45
46    if (!titulo) {
47      newErrors.titulo =
48        "El título es obligatorio.";
49    }
50
51    if (!descripcion) {
52      newErrors.descripcion =
53        "La descripción es obligatoria.";
54    }
55
56    setErrors(newErrors);
57
58    return Object.keys(newErrors).length === 0;
59  };
60
61  const handleSubmit = async (event) => {
62    event.preventDefault();
63
64    setMessage("");
65    setMessageType("");
66
67    if (!validateForm()) {
68      return;
69    }
70
71    try {
72      setIsSubmitting(true);
73
74      const result = await createTicket({
75        titulo: normalizeText(formData.titulo),
76        descripcion: normalizeText(formData.descripcion)
77      });
78
```

```
      setMessageType("success");
      setMessage(result.message);

      setFormData({
        titulo: "",
        descripcion: ""
      });

      setErrors({});
    } catch (error) {
      console.error(error);

      setMessageType("error");
      setMessage(
        "No se pudo registrar el ticket."
      );
    } finally {
      setIsSubmitting(false);
    }
  };

```

```
100     return (
101       <MainLayout>
102         <PageContainer title="Registro de Tickets">
103           <p className="text-gray-600 mb-6">
104             Complete la información de la incidencia técnica.
105           </p>
106
107           <form onSubmit={handleSubmit}>
108             <InputField
109               label="Título del ticket"
110               name="titulo"
111               value={formData.titulo}
112               placeholder="Ejemplo: Equipo no inicia"
113               required={true}
114               onChange={handleChange}
115               error={errors.titulo}
116             />
117
118             <div className="mb-4">
119               <label
120                 htmlFor="descripcion"
121                 className="
122                   block
123                   mb-2
124                   font-medium
125                   text-gray-700
126                 "
127               >
128                 Descripción de la incidencia
129               </label>
130
```

```
131                 <textarea
132                   id="descripcion"
133                   name="descripcion"
134                   value={formData.descripcion}
135                   placeholder="Describa detalladamente el problema"
136                   rows="5"
137                   onChange={handleChange}
138                   className={`
139                     w-full
140                     border
141                     rounded
142                     px-3
143                     py-2
144                     outline-none
145                     focus:ring-2
146                     ${
147                       errors.descripcion
148                         ? "border-red-500 focus:ring-red-300"
149                         : "border-gray-300 focus:ring-blue-300"
150                     }
151                   `}
152                 />
153
154                 {errors.descripcion && (
155                   <p className="text-red-500 text-sm mt-1">
156                     {errors.descripcion}
157                   </p>
158                 )}
159               </div>
160
```

```
            <Button
              type="submit"
              disabled={isSubmitting}
            >
              {isSubmitting
                ? "Registrando..."
                : "Registrar ticket"}
            </Button>
          </form>

          <FormMessage
            type={messageType}
            message={message}
          />
        </PageContainer>
      </MainLayout>
    );
  }
```

Explicación

Esta función:

- Elimina espacios innecesarios;
- Evita registros inconsistentes;
- Mejora calidad de datos.

Paso 8. Verificar App.js

Ruta exacta:

frontend/src/App.jsx

Contenido esperado

App.jsx

frontend > src > App.jsx > default

```jsx
import CreateTicket from "./pages/CreateTicket";

function App() {
  return <CreateTicket />;
}

export default App;
```

Paso 9. Verificar index.css

Ruta exacta

frontend/src/index.css

Contenido esperado

index.css 3

frontend > src > # index.css > body

```css
@tailwind base;
@tailwind components;
@tailwind utilities;

body {
  margin: 0;
  padding: 0;
  font-family: Arial, Helvetica, sans-serif;
  background-color: #f3f4f6;
}
```

Paso 10. Ejecutar frontend

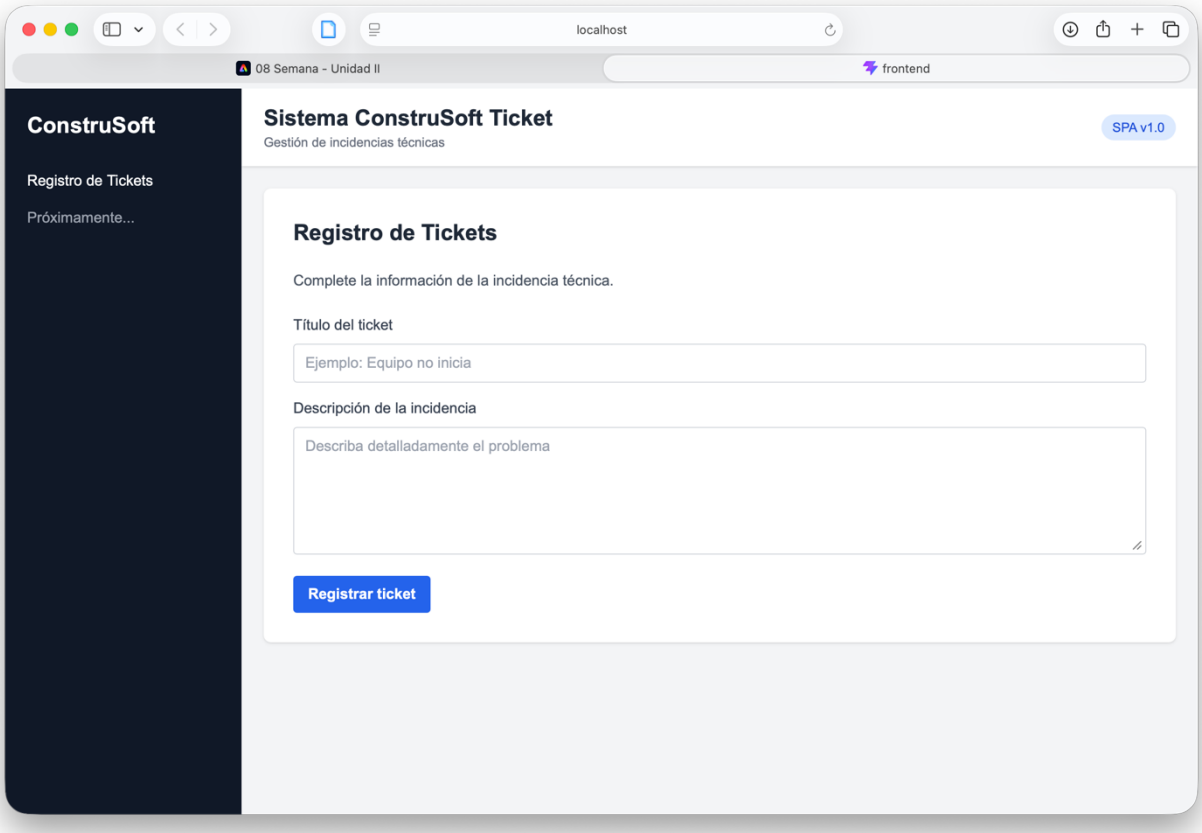
Comado

npm run dev

Paso 11. Resultado esperado

La aplicación debe mostrar:

- Sidebar lateral
- Cabecera superior
- Formulario integrado
- Diseño empresarial
- Estructura spa
- Validaciones visuales
- Formulario funcional



Paso 12. Validar persistencia

Registrar un ticket válido.

Resultado esperado

- Mensaje visual de éxito;
- Registro almacenado en postgresql;

- Formulario limpio;
- Interfaz organizada.

9. Explicación técnica

¿Qué se refinó realmente?

Elemento	Mejora
Layout	Separado del contenido
Componentes	Más reutilizables
Organización	Carpeta layouts
UX	Estructura empresarial
Escalabilidad	Preparada para nuevas páginas
Mantenibilidad	Menor acoplamiento

¿Por qué esto es importante?

Porque los sistemas reales:

- Crecen;
- Cambian;
- Incorporan nuevos módulos.

Si no existe refinamiento:

El frontend se vuelve inmanejable.

10. Errores frecuentes

Error	Causa	Solución
Tailwind no aplica	Frontend no reiniciado	Ejecutar npm run dev
Pantalla blanca	Error de importación	Revisar consola
Sidebar no aparece	MainLayout no importado	Verificar rutas
Formulario desaparece	children faltante	Revisar MainLayout.jsx
Error JSX	Etiquetas mal cerradas	Revisar VSCode

11. Resultado esperado de la semana

El sistema debe:

- Verse como SPA empresarial
- Mantener validaciones

- Mantener persistencia
- Tener frontend organizado
- Reutilizar layouts
- Reutilizar componentes
- Mantener Clean Architecture

12. Criterios de evaluación docente

Criterio	Indicador
Refinamiento	Código más organizado
Arquitectura SPA	Layout reutilizable
UX/UI	Interfaz empresarial
Reutilización	Componentes desacoplados
Persistencia	Flujo intacto
Mantenibilidad	Estructura preparada para crecer

“El refinamiento de código permite que un sistema evolucione sin convertirse en un problema para quienes lo desarrollan y mantienen.”

UPLA

Construcción de software

Semana 09
(Laboratorio)

Identificación y diseño de módulos de salida de datos en el Sistema ConstruSoft Ticket

ASIGNATURA

Construcción de software

ACTIVIDAD 01

“La información solo genera valor cuando puede transformarse en conocimiento útil para la toma de decisiones.”
— Peter Drucker

1. **Objetivo**
   Analizar y diseñar el primer módulo de salida de datos del sistema ConstruSoft Ticket, identificando los principios de construcción de módulos de salida y proponiendo una solución que permita visualizar información útil para los usuarios del sistema.

2. **Contexto**
   Durante las semanas anteriores se construyó progresivamente el sistema ConstruSoft Ticket, implementando:
   - Arquitectura limpia (Clean Architecture);
   - Persistencia con postgresql;
   - Validaciones en frontend y backend;
   - Reutilización de componentes;
   - Refinamiento de código;
   - Estructura SPA.

   Actualmente el sistema permite registrar incidencias técnicas correctamente, pero presenta una limitación importante:

   La información almacenada no puede ser consultada ni visualizada por los usuarios.

   En sistemas empresariales no basta con capturar información; también es necesario presentarla de forma clara, organizada y útil para apoyar la toma de decisiones.

   Por ello, la siguiente etapa del proyecto consiste en diseñar los módulos de salida que permitan transformar los datos almacenados en información accesible para los usuarios.

3. **Problema de la semana**

   ¿Cómo diseñar un módulo de salida de datos que permita visualizar la información registrada en el sistema ConstruSoft Ticket de manera clara, organizada y útil para los usuarios?

4. **Condiciones y restricciones**

   El estudiante deberá considerar:

   - Mantener la arquitectura implementada;
   - Respetar la estructura spa desarrollada;
   - Reutilizar componentes cuando sea posible;
   - Diseñar la solución sin implementar todavía toda la funcionalidad;
   - Identificar claramente los elementos necesarios para construir el módulo de salida.

5. **Actividades / Preguntas orientadoras**

   1. Análisis de los datos almacenados

   - ¿Qué información registra actualmente el sistema?
   - ¿Qué datos serían relevantes para visualizar?
   - ¿Qué información sería útil para un responsable de soporte técnico?

   2. Principios de construcción de módulos de salida

   - ¿Qué características debe tener una salida de datos efectiva?
   - ¿Qué diferencia existe entre mostrar datos e información?
   - ¿Cómo influye el diseño visual en la comprensión de los datos?

   3. Diseño del módulo de salida

   Diseñe una propuesta para visualizar los tickets registrados considerando:

   - Identificador;
   - Título;
   - Descripción;
   - Estado;
   - Fecha de creación.

4. Diseño arquitectónico

Analice qué componentes serían necesarios para implementar el módulo de salida:

Backend

- Endpoint GET
- DTO de respuesta
- Método de consulta

Frontend

- Página de listado
- Tabla de datos
- Componentes reutilizables
- Navegación SPA

5. Diseño de experiencia de usuario

- ¿Cómo debería integrarse el listado dentro de la SPA?
- ¿Dónde debería ubicarse el acceso al listado?
- ¿Qué elementos visuales mejorarían la lectura de la información?

6. **Entregable**

El estudiante deberá presentar:

Documento de análisis

Que contenga:

Análisis del módulo de salida

- Información a mostrar;
- Usuarios beneficiados;
- Utilidad de la información.

Diseño de la solución

- Estructura de pantalla;
- Componentes requeridos;

- Propuesta de navegación.

Diseño arquitectónico

- Endpoint propuesto;
- Dto de salida;
- Componentes frontend.

Evidencias

- Diagramas;
- Bosquejos;
- Prototipos;
- Capturas de diseño.

7. **Reflexión final**

Responder:

1. ¿Por qué es importante diseñar adecuadamente los módulos de salida?
2. ¿Qué riesgos existen al presentar información desorganizada?
3. ¿Cómo influye la calidad de las salidas en la toma de decisiones?
4. ¿Por qué los módulos de salida deben diseñarse antes de implementarse?

“Un sistema no aporta valor por la cantidad de datos que almacena, sino por la capacidad de convertirlos en información útil para las personas.”

— Jorge A. Vega Flores

Semana 09
Solución (modelo)
Identificación y diseño de módulos de salida de datos en el Sistema ConstruSoft Ticket

1. Propósito de la actividad

Orientar el desarrollo de la Semana 09 mediante el análisis y diseño del primer módulo de salida de datos del sistema ConstruSoft Ticket, preparando la arquitectura necesaria para que en la Semana 10 se implemente la visualización de tickets registrados.

Esta semana es particularmente importante porque marca la transición entre:

Unidad II - Captura de información

hacia

Unidad III - Visualización de información

El estudiante debe comprender que:
Un sistema no genera valor únicamente por almacenar datos; genera valor cuando esos datos pueden convertirse en información útil.

2. Estado esperado antes de iniciar

El proyecto debe contar con:

Componente	Estado esperado
Backend	ASP.NET Core operativo
Frontend	React + Vite operativo
PostgreSQL	Base de datos funcional
Arquitectura	Clean Architecture
SPA	Sidebar + Header + Layout
Formulario	Registro de tickets operativo
Persistencia	Tickets almacenados correctamente

3. Objetivo técnico de la semana

Diseñar el primer módulo de salida de datos:
Listado de Tickets
identificando:

- Datos a mostrar;
- Componentes necesarios;
- Endpoint requerido;
- Dtos de salida;
- Integración spa;
- Estructura visual.

Importante:
En esta semana NO implementaremos aún el endpoint GET ni el listado funcional.
La finalidad es diseñar la solución para posteriormente implementarla en la Semana 10.

4. **Resultado esperado**
   Al finalizar la sesión deberá existir:

   - Diseño del módulo de salida.
   - Estructura arquitectónica definida.
   - Componentes identificados.
   - Navegación planificada.
   - Prototipo de interfaz.
   - Dtos definidos.

5. **Desarrollo paso a paso**
   Paso 1. Verificar proyecto funcional
   Backend
   Abrir terminal.

   macOS
   cd ~/Documents/ConstruSoftTicket/backend/ConstruSoftTicket.API
   dotnet run

   Windows
   cd C:\ConstruSoftTicket\backend\ConstruSoftTicket.API
   dotnet run

   Resultado esperado
   http://localhost:5153

   Paso 2. Verificar frontend
   Abrir nueva terminal.
   macOS
   cd ~/Documents/ConstruSoftTicket/frontend
   npm run dev

   Windows
   cd C:\ConstruSoftTicket\frontend
   npm run dev

   Resultado esperado
   http://localhost:5173

6. **Analizar la información disponible**
   Paso 3. Revisar entidad Ticket
   Ruta exacta
   backend/ConstruSoftTicket.Domain/Entities/Ticket.cs

   Analizar atributos
   Identificar:
   Id
   Titulo
   Descripcion
   Estado
   FechaCreacion

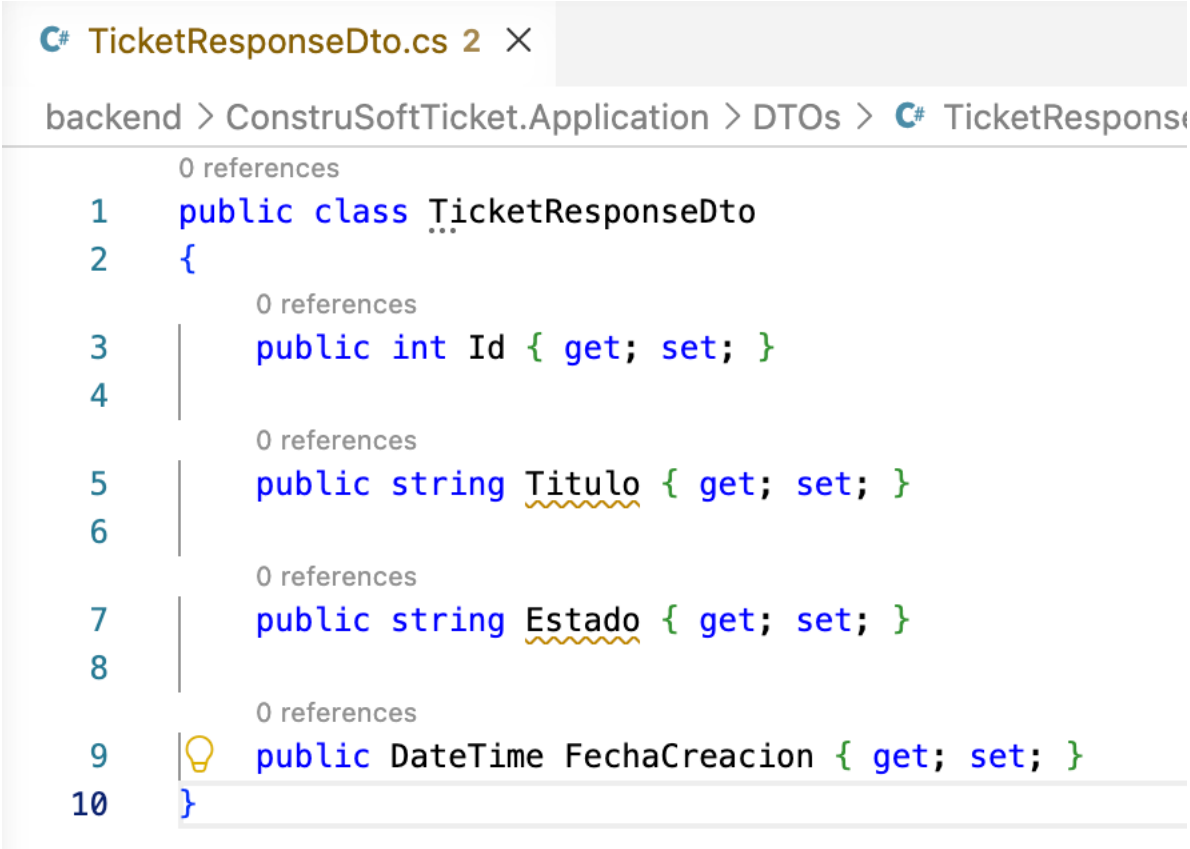
   Pregunta:
   ¿Cuáles de estos campos deberían mostrarse en un listado?

   Respuesta esperada

Campo	Mostrar
Id	Sí
Titulo	Sí
Descripcion	Opcional
Estado	Sí
FechaCreacion	Sí

7. **Diseñar el DTO de salida**
   Paso 4. Crear propuesta DTO
   Ruta futura
   backend/ConstruSoftTicket.Application/DTOs/TicketResponseDto.cs

   Diseñar (NO crear todavía)



Explicación docente
Un DTO de salida:

- Evita exponer entidades directamente;
- Mejora seguridad;
- Facilita mantenimiento;
- Permite controlar qué información se muestra.

8. **Diseñar el endpoint**
   Paso 5. Identificar endpoint requerido
   Actualmente existe:
   POST /api/ticket

   Pregunta
   ¿Qué endpoint necesitamos para consultar información?

   Respuesta esperada
   GET /api/ticket

   Diseño esperado
   GET /api/ticket

   Respuesta

```
{
  "success": true,
  "data": []
}
```

9. **Diseñar componentes frontend**

Paso 6. Analizar estructura SPA actual

Ruta
frontend/src

Estructura actual

components/
layouts/
pages/
services/
utils/

Pregunta

¿Qué nuevos componentes necesitamos?

Respuesta esperada

TicketList.jsx
TicketTable.jsx
TicketRow.jsx
StatusBadge.jsx

Paso 7. Diseñar TicketList

Ruta futura
frontend/src/pages/TicketList.jsx

Responsabilidad

- Solicitar datos a la API;
- Controlar estados;
- Mostrar tabla.

Paso 8. Diseñar TicketTable

Ruta futura
frontend/src/components/TicketTable.jsx

Responsabilidad

Mostrar:

ID	Título	Estado	Fecha

Paso 9. Diseñar StatusBadge
Ruta futura
frontend/src/components/StatusBadge.jsx

Responsabilidad
Mostrar estados visuales:
Abierto
En Proceso
Cerrado

10. **Diseñar integración SPA**
    Paso 10. Analizar Sidebar
    Ruta actual
    frontend/src/components/Sidebar.jsx

    Estado actual
    Registro de Tickets

    Diseño esperado
    Registro de Tickets
    Listado de Tickets

    Actividad docente
    Solicitar al estudiante diseñar:
    - Ubicación;
    - Navegación;
    - Flujo.

11. **Diseñar interfaz visual**
    Paso 11. Elaborar prototipo
    Diseño sugerido

12. **Diseño arquitectónico final**
    Paso 12. Completar matriz de diseño

Capa	Elemento
API	GET /api/ticket
Application	GetAllTickets()
DTO	TicketResponseDto
Infrastructure	Consulta PostgreSQL
React	TicketList

React	TicketTable
React	StatusBadge
SPA	Sidebar actualizado